

# Web Applications and Services

Course Study Guide

---

João Pavão

University of Trás-os-Montes e Alto Douro

# **Web Applications and Services**

**Course Textbook**

João Pavão

2026-08-31

# Table of contents

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Preface</b>                                                                | <b>1</b>  |
| Organisation . . . . .                                                        | 1         |
| Source and Updates . . . . .                                                  | 1         |
| How to Use This Textbook . . . . .                                            | 2         |
| Author . . . . .                                                              | 2         |
| <b>1. Web Application Architecture</b>                                        | <b>3</b>  |
| 1.1. The Problem: Building a Web Application from Scratch . . . . .           | 3         |
| 1.2. Two Orientations: Presentation vs. Service . . . . .                     | 3         |
| 1.3. The MVC Pattern . . . . .                                                | 4         |
| 1.4. Three Pieces: Frontend, Middleware and Backend . . . . .                 | 4         |
| 1.5. The Client-Server Model . . . . .                                        | 5         |
| <b>2. HTML, CSS and Bootstrap</b>                                             | <b>6</b>  |
| 2.1. The Main HTML Tags . . . . .                                             | 6         |
| 2.2. HTML Structure . . . . .                                                 | 7         |
| 2.3. Applying Styles Directly . . . . .                                       | 10        |
| 2.3.1. Three Ways of Applying CSS . . . . .                                   | 10        |
| 2.3.2. Classes, IDs and Specificity . . . . .                                 | 11        |
| 2.4. Applying Styles Using Frameworks . . . . .                               | 13        |
| 2.4.1. Bootstrap . . . . .                                                    | 13        |
| 2.4.2. Other Frameworks . . . . .                                             | 15        |
| 2.5. Best Practices . . . . .                                                 | 16        |
| 2.6. Exercise: the Static Pages of the “World Capitals” Application . . . . . | 16        |
| <b>3. The HTTP Protocol</b>                                                   | <b>18</b> |
| 3.1. The Middleware Connecting Frontend and Backend . . . . .                 | 18        |
| 3.2. The TCP/IP Model . . . . .                                               | 18        |
| 3.3. HTTP . . . . .                                                           | 19        |
| 3.3.1. Methods . . . . .                                                      | 19        |
| 3.3.2. Message Format . . . . .                                               | 20        |
| 3.3.3. Status Codes . . . . .                                                 | 22        |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>4. Java EE and Servlet Fundamentals</b>                                       | <b>23</b> |
| 4.1. The Jakarta EE Platform . . . . .                                           | 23        |
| 4.2. Layered Architecture . . . . .                                              | 23        |
| 4.3. Why Servlets? . . . . .                                                     | 24        |
| 4.4. The Life Cycle of a Servlet . . . . .                                       | 25        |
| 4.5. From the HTTP Message to the Java Method . . . . .                          | 26        |
| 4.6. Content-Type and the Shape of the Response . . . . .                        | 27        |
| 4.7. A First Servlet: “World Capitals” . . . . .                                 | 28        |
| 4.8. Servlets and the MVC Pattern: the Missing View . . . . .                    | 29        |
| <b>5. The Stateful Model: Sessions and JSP</b>                                   | <b>31</b> |
| 5.1. HTTP Is Stateless: the Problem . . . . .                                    | 31        |
| 5.2. <i>Cookies</i> . . . . .                                                    | 31        |
| 5.3. Sessions ( <code>HttpSession</code> ) . . . . .                             | 32        |
| 5.4. Session-Based Authentication . . . . .                                      | 33        |
| 5.5. The MVC Pattern with <i>Servlet</i> and JSP . . . . .                       | 33        |
| 5.6. A Simple JSP Example . . . . .                                              | 34        |
| 5.7. Lifecycle and Implicit Objects . . . . .                                    | 35        |
| 5.8. <i>Scopes</i> : Sharing Information Within the Server . . . . .             | 35        |
| <b>6. Security and Filters</b>                                                   | <b>38</b> |
| 6.1. The Problem: Security Repeated in Every <i>Servlet</i> . . . . .            | 38        |
| 6.2. Authentication vs. Authorisation . . . . .                                  | 38        |
| 6.3. What a Filter Is . . . . .                                                  | 39        |
| 6.4. Where a Filter Sits in the Request . . . . .                                | 39        |
| 6.5. A Filter’s Lifecycle . . . . .                                              | 40        |
| 6.6. <i>Servlet</i> and Filter: Separation of Responsibilities . . . . .         | 41        |
| 6.7. Declaring and Implementing a Filter . . . . .                               | 41        |
| 6.8. Redirection vs. 401: an Architectural Difference . . . . .                  | 42        |
| 6.9. Preparing for Stateless Security . . . . .                                  | 43        |
| <b>7. REST Web Services and Stateless Architecture (Part 1)</b>                  | <b>44</b> |
| 7.1. The Problem: Applications That Need to Communicate . . . . .                | 44        |
| 7.2. What a Web Service Is . . . . .                                             | 44        |
| 7.3. SOAP and REST: Two Approaches . . . . .                                     | 45        |
| 7.4. REST: Fielding’s Principles . . . . .                                       | 45        |
| 7.5. Resource-Oriented Architecture . . . . .                                    | 45        |
| 7.6. HTTP Methods as the API’s Contract . . . . .                                | 46        |
| 7.7. JSON as a Resource’s Representation . . . . .                               | 46        |
| 7.8. From a Web Application to a REST Service . . . . .                          | 46        |
| 7.9. Stateful vs. Stateless . . . . .                                            | 47        |
| 7.10. Why the Absence of State Matters . . . . .                                 | 47        |
| <b>8. REST Web Services (Part 2) and the JavaScript Client</b>                   | <b>48</b> |
| 8.1. Session Limitations in Distributed Systems . . . . .                        | 48        |
| 8.2. Stateless Authentication: the <code>Authorization</code> Header . . . . .   | 48        |
| 8.3. JWT: a Stateless <i>Token</i> . . . . .                                     | 48        |
| 8.4. The Lifecycle of a <i>Token</i> . . . . .                                   | 49        |
| 8.5. Horizontal Scalability . . . . .                                            | 51        |
| 8.6. Distributed MVC: the Pattern Splits Across Server and Client . . . . .      | 51        |
| 8.7. Consuming a REST API in JavaScript: <code>fetch()</code> and JSON . . . . . | 52        |
| 8.8. The DOM as the <i>Browser</i> ’s “View” . . . . .                           | 52        |

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>Appendices</b>                                                                 | <b>54</b> |
| <b>A. Complete Code: Static “World Capitals” Pages</b>                            | <b>54</b> |
| A.1. Introduction . . . . .                                                       | 54        |
| A.2. Initial Project Structure . . . . .                                          | 54        |
| A.3. <code>index.html</code> : Country Listing . . . . .                          | 55        |
| A.4. <code>login.html</code> : Authentication . . . . .                           | 56        |
| A.5. <code>capital.html</code> : Capital Lookup . . . . .                         | 57        |
| A.6. <code>paises-editor.html</code> : Country Management . . . . .               | 58        |
| <b>B. JWT Token Signing</b>                                                       | <b>61</b> |
| B.1. Introduction . . . . .                                                       | 61        |
| B.2. HMAC: a Shared Secret Key . . . . .                                          | 61        |
| B.3. RSA and ECDSA: a Key Pair . . . . .                                          | 61        |
| B.4. Choosing Between HMAC and Asymmetric Signing . . . . .                       | 63        |
| B.5. The <code>alg:none</code> Attack . . . . .                                   | 63        |
| B.6. Where to Store the Token on the Client . . . . .                             | 64        |
| B.7. Minimum Best Practices . . . . .                                             | 64        |
| <b>C. Development Environment Setup</b>                                           | <b>66</b> |
| C.1. Introduction . . . . .                                                       | 66        |
| C.2. The Tools for This Course . . . . .                                          | 66        |
| C.3. Required Tools . . . . .                                                     | 67        |
| C.4. Confirming the Installation . . . . .                                        | 68        |
| <b>D. LAB1: First Endpoint and Simulated DAO</b>                                  | <b>69</b> |
| D.1. Introduction to the Practical LABs . . . . .                                 | 69        |
| D.2. LAB1: First Endpoint and Simulated DAO . . . . .                             | 69        |
| D.2.1. Step 1.1: Prepare the Development Environment . . . . .                    | 69        |
| D.2.2. Step 1.2: Create the <i>Dynamic Web Project</i> . . . . .                  | 70        |
| D.2.3. Step 1.3: Develop the First <i>Endpoint</i> ( <i>Servlet</i> ) . . . . .   | 70        |
| D.2.4. Step 1.4: Deploy and Test the Application . . . . .                        | 71        |
| D.2.5. Step 1.5: Implement the Database Interface via a DAO . . . . .             | 71        |
| D.2.6. Step 1.6: Create the Data Model (POJO) . . . . .                           | 72        |
| D.2.7. Step 1.7: Develop the Capitals <i>Endpoint</i> . . . . .                   | 72        |
| D.2.8. Step 1.8: Develop the Countries <i>Endpoint</i> . . . . .                  | 72        |
| <b>E. LAB2: Login and Session</b>                                                 | <b>73</b> |
| E.1. LAB2: Login and Session . . . . .                                            | 73        |
| E.1.1. Step 2.1: Create the <i>Login Endpoint</i> . . . . .                       | 73        |
| E.1.2. Step 2.2: Implement the Authentication Logic . . . . .                     | 73        |
| E.1.3. Step 2.3: Create and Manage the Session . . . . .                          | 74        |
| E.1.4. Step 2.4: Handle Authentication Failures . . . . .                         | 74        |
| E.1.5. Step 2.5: Create an HTML File to Test the <i>Login Endpoint</i> . . . . .  | 74        |
| E.1.6. Step 2.6: Protect an <i>Endpoint</i> with Session Verification . . . . .   | 75        |
| E.1.7. How to Test . . . . .                                                      | 75        |
| <b>F. LAB3: AuthenticationFilter and Logout</b>                                   | <b>76</b> |
| F.1. LAB3: AuthenticationFilter and Logout . . . . .                              | 76        |
| F.1.1. Step 3.1: Create and Configure the Authentication Filter . . . . .         | 76        |
| F.1.2. Step 3.2: Implement the Verification Logic in the Filter . . . . .         | 76        |
| F.1.3. Step 3.3: Remove the Manual Verification from the <i>Servlet</i> . . . . . | 77        |
| F.1.4. Step 3.4: Create the <i>Logout Endpoint</i> . . . . .                      | 77        |

## Table of contents

|           |                                                                                      |           |
|-----------|--------------------------------------------------------------------------------------|-----------|
| F.1.5.    | Step 3.5: Integrated Testing . . . . .                                               | 78        |
| F.1.6.    | Conclusion . . . . .                                                                 | 78        |
| <b>G.</b> | <b>LAB4: JDBC and Complete CRUD</b>                                                  | <b>79</b> |
| G.1.      | LAB4: JDBC and Complete CRUD . . . . .                                               | 79        |
| G.1.1.    | Step 4.1: Prepare the Database . . . . .                                             | 79        |
| G.1.2.    | Step 4.2: Configure JDBC and Update DBDAO . . . . .                                  | 79        |
| G.1.3.    | Step 4.3: Update the <i>Login</i> Endpoint . . . . .                                 | 81        |
| G.1.4.    | Step 4.4: Create a CRUD Endpoint Using POST and the operation<br>Parameter . . . . . | 81        |
| G.1.5.    | Step 4.5: Create HTML Forms for Testing . . . . .                                    | 82        |
| G.1.6.    | Step 4.6: Protect the Endpoints with the Filter . . . . .                            | 82        |
| G.1.7.    | Step 4.7: Integrated Testing . . . . .                                               | 83        |
| G.1.8.    | Conclusion . . . . .                                                                 | 83        |
| <b>H.</b> | <b>LAB5 (Optional): Stateless Authentication with JWT</b>                            | <b>84</b> |
| H.1.      | LAB5 (Optional): Stateless Authentication with JWT . . . . .                         | 84        |
| H.1.1.    | Stateful vs. Stateless . . . . .                                                     | 84        |
| H.1.2.    | Step 5.1: Add a JWT Library . . . . .                                                | 85        |
| H.1.3.    | Step 5.2: Create a Small JWT Utility Class . . . . .                                 | 85        |
| H.1.4.    | Step 5.3: Change <i>/login</i> to Return a Token (No Session) . . . . .              | 86        |
| H.1.5.    | Step 5.4: Create a JWT Authentication Filter . . . . .                               | 87        |
| H.1.6.    | Step 5.5: Remove the Session-Based Filter (or Disable It) . . . . .                  | 87        |
| H.1.7.    | Step 5.6: 401 vs. 403 in This Project . . . . .                                      | 88        |
| H.1.8.    | Step 5.7: How to Test ( <i>Browser</i> + <i>curl</i> /Postman) . . . . .             | 88        |
| H.1.9.    | Conclusion . . . . .                                                                 | 88        |
| <b>I.</b> | <b>LAB6: The Front-Office in JavaScript</b>                                          | <b>89</b> |
| I.1.      | LAB6: The Front-Office in JavaScript . . . . .                                       | 89        |
| I.1.1.    | Step 6.1: Prepare the Project . . . . .                                              | 89        |
| I.1.2.    | Step 6.2: A Small Addition to the <i>Backend</i> — Listing the Available Codes       | 90        |
| I.1.3.    | Step 6.3: Authentication Without Reloading the Page . . . . .                        | 91        |
| I.1.4.    | Step 6.4: Show/Hide Based on Authentication State . . . . .                          | 92        |
| I.1.5.    | Step 6.5: List Countries on the Home Page . . . . .                                  | 93        |
| I.1.6.    | Step 6.6: Look Up a Capital . . . . .                                                | 94        |
| I.1.7.    | Step 6.7: Country CRUD . . . . .                                                     | 95        |
| I.1.8.    | How to Test . . . . .                                                                | 96        |
| I.1.9.    | Conclusion . . . . .                                                                 | 96        |

# Preface

This textbook is a self-study resource for the theoretical component of **Web Applications and Services** (ASW), Master's in Electrical and Computer Engineering (MEEC), covering Web application architecture, best development practices, the HTTP protocol, HTML and CSS, server-side and client-side programming, database access, and REST Web Services, with Java EE as the case study.

## Organisation

The textbook is organised into three thematic blocks:

1. **Web Foundations** — client-server architecture, HTML, CSS and the HTTP protocol
2. **Classic Java EE** — Servlets, the stateful model and session management
3. **Web Services** — security and filters, and REST Web Services, including consuming an API from JavaScript in the browser

The practical LABs (the same as ESIGELEC's, with a JavaScript front-office added in LAB6) are described in Appendices D1 to D6, with the development environment setup in Appendix C.

## Source and Updates

The content of this study guide is based on the course's official syllabus and on materials from ESIGELEC (Java EE slides and LABs) and UTAD (HTTP, HTML/CSS/Bootstrap, Servlets), organised and adapted for the 2026/2027 academic year. The reference bibliography for the course is listed in each chapter and compiled in the `referencias.bib` file.

## **How to Use This Textbook**

Each chapter corresponds to one theoretical class of the course. Students are advised to read a chapter before the corresponding class, and to use this text as support for independent study, to review concepts and clarify doubts. Reading does not replace attendance in class, nor participation in the practical LABs.

## **Author**

**João Pavão** — UTAD, course lecturer, theoretical component



# 1.

## Web Application Architecture

### 1.1. The Problem: Building a Web Application from Scratch

A Web application is not simply a program that runs on a single computer: it is a program that answers requests coming from multiple clients, simultaneously, over the network. Built from scratch, with no supporting platform, an application of this kind would have to solve, on its own, a set of problems common to any system of this sort:

- Receiving and interpreting requests coming from the network
- Building and returning responses in the format expected by the client
- Handling multiple concurrent requests (*multi-threading*)
- Managing database connections
- Retaining information across successive requests from the same user
- Protecting resources from unauthorised access

Solving all of these problems again and again, application after application, would be a waste of effort. This is exactly why enterprise platforms exist, providing these services ready to use, and freeing developers to focus on the business logic of the application.

### 1.2. Two Orientations: Presentation vs. Service

A Web application can follow two distinct orientations, depending on the kind of response it produces:

**Presentation-oriented** Generates interactive Web pages (HTML, CSS and dynamic content) in response to each request. This is the traditional Web application model, meant to be consumed directly by a human user through a *browser*.

**Service-oriented** Implements the *endpoint* of a Web service, typically returning structured data in JSON or XML format instead of HTML. This model is meant to be consumed by another program: a mobile application, a *single-page application*, or another service.

This distinction largely maps onto the split between *front-end* (the part of the application that interacts directly with whoever uses it) and *back-end* (the part that processes logic and data). A presentation-oriented application tends to blend these two responsibilities into a

single server-side component; a service-oriented application separates them clearly, leaving the *front-end* entirely up to the client.

### 1.3. The MVC Pattern

Organising all of an application's code as a single undifferentiated block quickly becomes unmanageable: the code that processes the request, the code that accesses data, and the code that generates the response end up tangled together, hard to test or change independently. The *Model-View-Controller* (MVC) pattern solves this by splitting the application into three distinct responsibilities:

**Model** Represents the application's data and business logic, independently of how it is presented. In a Web application, the Model is typically responsible for accessing the database and returning already-validated information.

**View** Responsible for presenting the data to whoever requests it, without containing business logic. In a presentation-oriented application, the View generates HTML; in a service-oriented application, there is, in fact, no server-side View at all: the response is just structured data (JSON), and it is the client that decides how to present it.

**Controller** Receives the request, decides which operation to perform on the Model, and chooses the View (or response format) to return. The Controller is what bridges the incoming HTTP request and the application's logic.

This separation of responsibilities is one of the most recurring design principles throughout this subject. It shows up in its classic form when a server-side component acts as Controller and delegates presentation to another, which acts as View; and it shows up in a far less obvious form when the View stops existing on the server side and is instead rebuilt in the *browser*, from JSON data.

### 1.4. Three Pieces: Frontend, Middleware and Backend

Both the presentation/service distinction and the MVC pattern describe the same central idea, seen from different angles: a Web application is not a single block, but a set of pieces with distinct roles, communicating with one another. Throughout this course, those pieces always reduce to the same three:

**Frontend** The part of the application that runs on the client side (typically a *browser*), responsible for presenting information and collecting input from whoever uses it. It largely corresponds to the View in MVC.

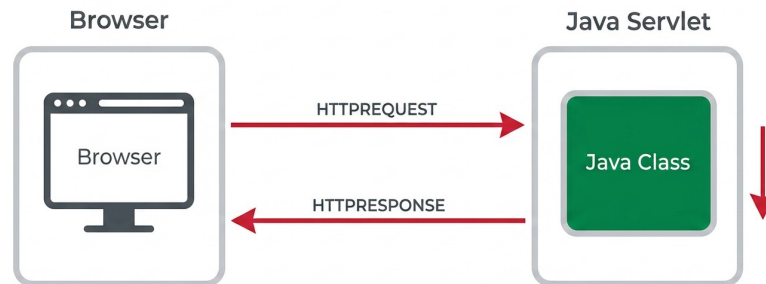
**Middleware** The protocol that connects the Frontend to the Backend, carrying requests and responses between the two. In this course, that protocol is always HTTP.

**Backend** The part of the application that runs on the server side, responsible for business logic and data access. It largely corresponds to the Model and the Controller in MVC.

This three-way division is the map for the rest of this book: first the Frontend is built ([Chapter 2](#), in HTML/CSS/Bootstrap), then the Middleware that connects it to the server is studied in detail ([Chapter 3](#), the HTTP protocol), and only then is the Backend built that actually answers those requests (from [Chapter 4](#) onward, with *Servlets* and, later, REST APIs).

## 1.5. The Client-Server Model

The Middleware connecting the Frontend to the Backend rests on the client-server model: the Frontend (typically a *browser*) sends a request over the network, and the Backend processes that request and returns a response.



**Figure 1.1.:** The client (for example, a browser) sends an HTTP request; the server processes it and returns a response. The exchange repeats at every interaction, without the server, by itself, retaining memory of the previous one.

This exchange takes place through the HTTP protocol (*HyperText Transfer Protocol*), which has two fundamental characteristics:

- It is transaction-oriented: each interaction consists of a request followed by a response
- It is *stateless*: the server does not, by itself, retain any memory of a client's previous requests

The absence of state at the protocol level does not prevent an application from needing to retain information across requests (for instance, to know that a user is authenticated). That need is addressed at the application level, not the protocol level, through mechanisms such as sessions and *cookies* (see [Chapter 5](#)).

The details of this protocol, the Middleware connecting these two pieces, are the subject of [Chapter 3](#).

# 2. HTML, CSS and Bootstrap

## 2.1. The Main HTML Tags

This chapter builds the first of the three pieces identified in [Chapter 1](#): the Frontend. It is the part of the application that runs in the *browser*, and it is the natural place to start, because it is the part that can be seen and tried out most immediately, even before any server exists behind it.

HTML (*HyperText Markup Language*) describes the content of a page through elements, each delimited by an opening and a closing *tag* (for example, `<p>` and `</p>`, delimiting a paragraph). Before any complete example, it is worth getting familiar with the vocabulary: the most commonly used *tags* for text content are these:

| Tag                                                | What it is for                                                                                                                                                                          | Example                                                                                           |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>&lt;h1&gt;</code> to <code>&lt;h6&gt;</code> | Headings and subheadings, from most important ( <code>&lt;h1&gt;</code> ) to least important ( <code>&lt;h6&gt;</code> ). Each page should have, at most, one <code>&lt;h1&gt;</code> . | <code>&lt;h1&gt;Page Title&lt;/h1&gt;</code>                                                      |
| <code>&lt;p&gt;</code>                             | A paragraph of text.                                                                                                                                                                    | <code>&lt;p&gt;This is a paragraph.&lt;/p&gt;</code>                                              |
| <code>&lt;a&gt;</code>                             | A link (hyperlink) to another page or section, defined by the <code>href</code> attribute.                                                                                              | <code>&lt;a href="https://www.w3.org"&gt;W3C&lt;/a&gt;</code>                                     |
| <code>&lt;ul&gt;</code>                            | An unordered list of items (with bullet points); each item goes inside an <code>&lt;li&gt;</code> .                                                                                     | <code>&lt;ul&gt;&lt;li&gt;Item1&lt;/li&gt;&lt;li&gt;Item2&lt;/li&gt;...&lt;/ul&gt;</code>         |
| <code>&lt;ol&gt;</code>                            | An ordered (numbered) list of items; like <code>&lt;ul&gt;</code> , items go inside <code>&lt;li&gt;</code> .                                                                           | <code>&lt;ol&gt;&lt;li&gt;First item&lt;/li&gt;&lt;li&gt;Second item&lt;/li&gt;&lt;/ol&gt;</code> |

To organise this content into blocks, HTML offers a generic container and several structural (or semantic) *tags*. The difference between them is that the generic container has no meaning of its own, while the semantic *tags* describe the role of each area of the page:

## 2. HTML, CSS and Bootstrap

| Tag                          | What it is for                                                                                                                 | Example                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>&lt;div&gt;</code>     | A generic container, with no meaning of its own; used to group other elements (for instance, to apply a common style to them). | <code>&lt;div class="card"&gt;...&lt;/div&gt;</code>                     |
| <code>&lt;header&gt;</code>  | The header of the page or of a section (typically title, logo, menu).                                                          | <code>&lt;header&gt;&lt;h1&gt;My Site&lt;/h1&gt;&lt;/header&gt;</code>   |
| <code>&lt;nav&gt;</code>     | A navigation block, typically a list of links.                                                                                 | <code>&lt;nav&gt;&lt;a href="#home"&gt;Home&lt;/a&gt;&lt;/nav&gt;</code> |
| <code>&lt;main&gt;</code>    | The main content of the page; there is, at most, one per page.                                                                 | <code>&lt;main&gt;&lt;p&gt;Content...&lt;/p&gt;&lt;/main&gt;</code>      |
| <code>&lt;section&gt;</code> | A thematic content section, usually with its own heading.                                                                      | <code>&lt;section&gt;&lt;h2&gt;About&lt;/h2&gt;&lt;/section&gt;</code>   |
| <code>&lt;aside&gt;</code>   | Related, but secondary, content (sidebar, notes).                                                                              | <code>&lt;aside&gt;&lt;p&gt;See also...&lt;/p&gt;&lt;/aside&gt;</code>   |
| <code>&lt;footer&gt;</code>  | The footer of the page or of a section (copyright, contacts).                                                                  | <code>&lt;footer&gt;© 2026&lt;/footer&gt;</code>                         |
| <code>&lt;form&gt;</code>    | Used to build data-entry forms, together with <i>input</i> fields.                                                             | <code>&lt;form&gt;&lt;input type="text"/&gt;&lt;/form&gt;</code>         |

### Reference for further study

This table covers only the *tags* used most often throughout this course. For the complete list of HTML elements, and the detailed specification of each, use the [W3C](#) (*World Wide Web Consortium*) reference, the international body that standardises HTML.

## 2.2. HTML Structure

### A note on the code examples in this chapter

Several examples below are followed by a screenshot of the page as rendered in the *browser*. To keep every screenshot consistent with the code shown above it, the on-screen text in those specific examples (headings, menu labels, form fields) is left exactly as it appears in the original Portuguese screenshots, even in this English edition. Examples not tied to a screenshot use English text throughout.

A minimal HTML document always has the same shape: a document type declaration, followed by an `html` element, split into a **head** (metadata, not visible to the public, only to developers) and a **body** (the content visible to the public):

```
1 <!DOCTYPE html>
2 <html lang="pt">
3   <head>
4     <meta charset="UTF-8">
```

```

5     <title>A Minha Página</title>
6 </head>
7 <body>
8     <h1>Bem-vindo</h1>
9     <p>Primeira versão da página.</p>
10 </body>
11 </html>

```

This code renders in the *browser* as shown in figure Figure 2.1:



Bem-vindo

Primeira versão da página.

**Figure 2.1.:** The minimal HTML page, as displayed in the browser: only the title and the paragraph from the `body` are visible.

Combining the structural *tags* already introduced, a page gains an organisation far clearer than a single block of undifferentiated `divs`. Each *tag* labels the kind of information it holds:

```

1 <!DOCTYPE html>
2 <html lang="pt">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>Página de Exemplo</title>
7 </head>
8 <body>
9     <header>
10         <h1>Título do Site</h1>
11         <!-- other elements in the header -->
12     </header>
13     <nav><!-- menu here --></nav>
14     <main>
15         <section>
16             <h2>Introdução</h2>
17             <p>Conteúdo inicial.</p>
18         </section>
19         <section>
20             <h2>Outra secção</h2>
21             <p>Podem colocar-se quantas secções se quiser.</p>
22         </section>
23     </main>
24     <footer>© 2026</footer>
25 </body>
26 </html>

```

This code renders in the *browser* as shown in figure Figure 2.2 — notice that, with no CSS at all, the structural *tags* carry no visual style of their own; the organisation into blocks exists, but is not yet visible:

# Título do Site

## Introdução

Conteúdo inicial.

## Outra secção

Podem colocar-se quantas secções se quiser.

© 2026

**Figure 2.2.:** The semantic page above, with no CSS applied: the **header**, the **nav** and the **sections** inside **main** organise the content, but are not visually distinguishable from one another.

Two elements are used with particular frequency in a Web application: a list of links inside **nav**, for navigation, and a **form**, to collect data from whoever uses the page.

```
1 <nav>
2   <ul>
3     <li><a href="#home">Início</a></li>
4     <li><a href="#sobre">Sobre</a></li>
5     <li><a href="#contacto">Contacto</a></li>
6   </ul>
7 </nav>
```

This code renders in the *browser* as shown in figure Figure 2.3: with no CSS, a list of links still looks like a list, with bullet points and the *browser*'s default blue, underlined formatting for links:

- [Início](#)
- [Sobre](#)
- [Contacto](#)

**Figure 2.3.:** The navigation list above, as displayed in the browser by default, with no CSS at all.

```
1 <section>
2   <h2>Formulário de contacto</h2>
3   <form>
4     <label for="nome">Nome</label>
5     <input id="nome" name="nome" type="text">
6     <label for="email">Email</label>
7     <input id="email" name="email" type="email">
8     <button type="submit">Enviar</button>
```

```

9     </form>
10  </section>

```

This code renders in the *browser* as shown in figure Figure 2.4: once again, with no CSS, the fields and the button appear with the *browser's* default styling:

## Formulário de contacto

Nome  Email

**Figure 2.4.:** The contact form above, as displayed in the browser by default, with no CSS at all.

Note the `for` attribute of each `label`, which matches the `id` of the corresponding `input`: this is what visually links a label to its field, and what allows a screen reader to correctly announce the form. The `input` tag is one of the few tags that is not used with an opening and a closing pair: `<input> </input>` — it only has the form `<input .../>`, whose trailing part means that this tag “opens and closes” itself. Most of the time, that trailing `/>` is omitted and only `>` is written, as shown in the code examples above.

## 2.3. Applying Styles Directly

HTML describes the **structure of the page**; the **presentation** is the responsibility of [CSS](#) (*Cascading Style Sheets*). This section shows the three ways of attaching CSS to HTML, and then the two main selectors — class and identifier — used to target those rules at specific elements.

### 2.3.1. Three Ways of Applying CSS

CSS can be attached to HTML in three distinct ways.

**Inline**, directly in an element’s `style` attribute. For example, on an `h1` element:

```

1 <h1 style="color:#0d6efd; margin:0.5rem 0;">Título do Site</h1>

```

This code renders in the *browser* as shown in figure Figure 2.5:

# Título do Site

**Figure 2.5.:** The heading above, already coloured blue by the inline style.

**Internal**, in a `<style>` block inside the `<head>` of the `html` file it refers to:



```

1 <head>
2   <meta charset="UTF-8">
3   <style>
4     :root { --brand: #0d6efd; }
5     body { font-family: system-ui, sans-serif; line-height:1.5; }
6     nav ul { list-style:none; display:flex; gap:1rem; padding:0; }
7     nav a { text-decoration:none; padding:0.4rem 0.6rem; }
8     nav a:hover { background:var(--brand); }
9   </style>
10 </head>

```

The rule `:root { --brand: #0d6efd; }` defines a CSS variable (a *custom property*), reused later with `var(--brand)`: it is useful for keeping the same colour in several places in the style sheet without repeating it, and for being able to change it in a single place.

**External**, in a separate `.css` file, linked through the `<link>` tag placed in the header of the document where the styles should apply:

```

1 <head>
2   <meta charset="UTF-8">
3   <link rel="stylesheet" href="styles.css">
4 </head>

```

```

1 /* styles.css */
2 :root { --brand: #0d6efd; }
3 body { font-family: system-ui, sans-serif; line-height:1.5; }
4 nav ul { list-style:none; display:flex; gap:1rem; padding:0; }
5 nav a { text-decoration:none; padding:0.4rem 0.6rem; }
6 nav a:hover { background: var(--brand); }

```

Applying these rules (through any of the three methods) to the navigation list shown earlier, the result is shown in figure Figure 2.6: the list's bullet points disappear, the links stop being underlined, and they line up side by side:

Início      Sobre      Contacto

**Figure 2.6.:** The same navigation list, now with the CSS applied (`list-style:none`, `display:flex`, `text-decoration:none`).

Of the three, external CSS is, in general, the best practice: it clearly separates structure (HTML) from presentation (CSS), allows the same file to be reused across several pages, and keeps each document shorter and easier to read.

### 2.3.2. Classes, IDs and Specificity

Once the structure is defined in HTML, the CSS will hold a set of instructions telling the HTML elements how they should be “presented”. To apply styles to elements, CSS uses two main selectors: the class (`class`), for groups of elements, and the identifier (`id`), which must be unique within the page. A class can therefore label several elements on a page, which will

all be formatted identically. Any number of elements can share the same class within the same document. An id selector, on the other hand, should only ever be applied to a single element on a page. The id is therefore said to be more specific than the `class`.

When writing CSS rules for a class, a dot is placed before the class name: `.class_name`, followed by the formatting instructions between `{ }`, each separated by a semicolon. On the HTML side, the elements meant to be formatted by those rules carry the attribute `class="class_name"`. For HTML elements with an `id="id_name"` attribute, formatting works the same way as for classes, except that instead of starting with `.`, the selector starts with `#` (`#id_name`).

A single HTML element can have several classes applied at once, separated by spaces. In that case, the element is formatted using the rules of ALL the applied classes:

```
1 <section class="noticias destaque com_hero"> conteúdo da secção </section>
```

Consider the following examples:

```
1 <h1 id="titulo" class="titulo">Título do Site</h1>
2 <nav class="menu">
3   <ul>
4     <li><a class="menu__link ativo" href="#home">Início</a></li>
5     <li><a class="menu__link" href="#sobre">Sobre</a></li>
6     <li><a class="menu__link" href="#contacto">Contacto</a></li>
7   </ul>
8 </nav>
```

```
1 :root { --brand: #0d6efd; }
2 #titulo { color: var(--brand); } /* id: unique per page */
3 .menu { border-block: 1px solid #eee; padding: 0.75rem 0; }
4 .menu__link { text-decoration: none; padding: 0.4rem 0.6rem; }
5 .menu__link.ativo { font-weight: 600; }
```

This code renders in the *browser* as shown in figure Figure 2.7: the heading is coloured by the `#titulo` rule, and the menu is styled by the `.menu/.menu__link` rules, with the `ativo` link shown in bold:

# Título do Site

Início Sobre Contacto

**Figure 2.7.:** The heading and menu above, now styled by the class and identifier rules.

When the same HTML element is styled by two conflicting CSS rules — for instance, both setting its colour — the rule that wins is determined by its specificity. The list of specificities, from highest to lowest, is:

1. `!important` (best avoided; makes the CSS hard to maintain)
2. Inline style, i.e., inside each element (`style="..."`)
3. Identifier (`#id`)

4. Class (`.class`), attribute or pseudo-class
5. Element (`h1`, `p`, `a`, ...)

For example, what colour would the heading from the previous example end up with, given this CSS?

```
1 h1 { color: black; }
2 .titulo { color: blue; }
3 #titulo { color: green; } /* wins over the two rules above */
```

The identifier (`#titulo`) is more specific than both the class and the element, so it wins: the heading turns green, as shown in figure Figure 2.8:

# Título do Site

**Figure 2.8.:** The heading above in green: the `#titulo` rule wins over the `h1` and `.titulo` rules, for being more specific.

Or, if `!important` is applied:

```
1 h1 { color: black !important; } /* wins because it has higher specificity */
2 .titulo { color: blue; }
3 #titulo { color: green; }
```

Now it is the element rule (`h1`) that wins, despite being the least specific of the three, because it carries `!important`: the heading turns black, as shown in figure Figure 2.9:

# Título do Site

**Figure 2.9.:** The heading above in black: the `h1` rule wins over the `.titulo` and `#titulo` rules, despite being the least specific, because it carries `!important`.

## 2.4. Applying Styles Using Frameworks

### 2.4.1. Bootstrap

Writing and maintaining all of an application's CSS from scratch is a considerable amount of work, especially when the result is expected to be responsive (adjusting to different screen sizes) and visually consistent. A CSS *framework* such as [Bootstrap](#) solves this problem, providing:

- Productivity and visual consistency, through ready-to-use utility classes
- Responsiveness already built in, through a *grid* and spacing classes
- Tested, accessible components (forms, navigation bars, buttons, among others)

## 2. HTML, CSS and Bootstrap

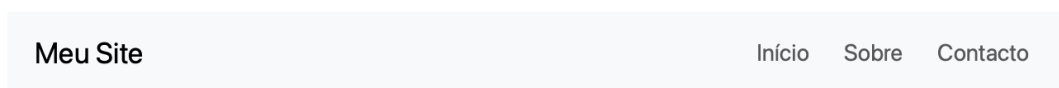
*Bootstrap* can be included directly via *CDN* (*Content Delivery Network*), with no installation required: a style sheet in the `<head>`, and a JavaScript file (for the interactive components) before the closing `</body>`.

```
1 <head>
2   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" re
3 </head>
4 <body>
5   <!-- page content -->
6   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
7 </body>
```

A navigation bar (*navbar*), with *Bootstrap*, is ready in a handful of lines, already responsive (it collapses into a menu on narrow screens):

```
1 <nav class="navbar navbar-expand-md bg-light border-bottom">
2   <div class="container">
3     <a class="navbar-brand" href="#">Meu Site</a>
4     <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target
5       <span class="navbar-toggler-icon"></span>
6     </button>
7     <div id="nav1" class="collapse navbar-collapse">
8       <ul class="navbar-nav ms-auto gap-2">
9         <li class="nav-item"><a class="nav-link" href="#home">Início</a></li>
10        <li class="nav-item"><a class="nav-link" href="#sobre">Sobre</a></li>
11        <li class="nav-item"><a class="nav-link" href="#contato">Contato</a></li>
12      </ul>
13    </div>
14  </div>
15 </nav>
```

This code renders in the *browser* as shown in figure Figure 2.10:



**Figure 2.10.:** The navigation bar above, already styled by Bootstrap.

A form gets the same visual treatment through the `form-label` and `form-control` classes:

```
1 <section class="container my-4">
2   <h2>Formulário de contacto</h2>
3   <form class="row g-3 col-lg-6">
4     <div class="col-12">
5       <label for="nome" class="form-label">Nome</label>
6       <input id="nome" name="nome" type="text" class="form-control">
7     </div>
8     <div class="col-12">
9       <label for="email" class="form-label">Email</label>
10      <input id="email" name="email" type="email" class="form-control">
```

```

11     </div>
12     <div class="col-12">
13         <button class="btn btn-primary" type="submit">Enviar</button>
14     </div>
15 </form>
16 </section>

```

This code renders in the *browser* as shown in figure Figure 2.11: compare it with figure Figure 2.4, the same form without Bootstrap:

## Formulário de contacto

Nome

Email

Enviar

**Figure 2.11.:** The contact form above, already styled by Bootstrap (`form-label`, `form-control`, `btn btn-primary`).

The same grid system also organises the main content into columns. Bootstrap's grid divides each row (row) into twelve columns; the class `col-6` occupies six of those twelve, i.e., half the width:

```

1 <main class="container my-4">
2   <div class="row">
3     <div class="col-6">
4       <p>Conteúdo principal...</p>
5     </div>
6     <aside class="col-6">
7       <p>Barra lateral...</p>
8     </aside>
9   </div>
10 </main>

```

This code renders in the *browser* as shown in figure Figure 2.12: the main content and the sidebar sit side by side, each occupying half the width, even though there is no visible border marking the columns — it is Bootstrap's grid that reserves the space for each one:

Conteúdo principal...

Barra lateral...

**Figure 2.12.:** The main content and the sidebar organised by Bootstrap's grid.

### 2.4.2. Other Frameworks

*Bootstrap* is not the only CSS framework available, only the most beginner-friendly to start with, thanks to its approach based on ready-to-use components. It is worth knowing about,

## 2. HTML, CSS and Bootstrap

even without going into depth, a few popular alternatives:

**Tailwind CSS** A *utility-first framework*: instead of ready-made components such as `navbar` or `card`, it provides low-level classes, one per CSS property (for example `flex`, `pt-4`, `text-center`), which are combined directly in the HTML to build any custom look.

**Bulma** Based purely on CSS, with no JavaScript at all; syntax close to *Bootstrap*'s, but with no ready-made interactive components (menus, modals).

**Materialize** Implements Google's *Material Design* (shadows, elevation, vivid colours, animations); visually more opinionated than *Bootstrap*.

**Foundation** Aimed at larger, more customised projects; more flexible than *Bootstrap*, but with a steeper learning curve.

The following table compares these *frameworks* by their approach and by ease of learning:

| Framework           | Approach                                                                             | Learning curve |
|---------------------|--------------------------------------------------------------------------------------|----------------|
| <b>Bootstrap</b>    | Ready-made components ( <code>navbar</code> , <code>card</code> , <code>btn</code> ) | Low            |
| <b>Tailwind CSS</b> | Utility classes, one per CSS property                                                | Medium/high    |
| <b>Bulma</b>        | CSS components, no JavaScript                                                        | Low            |
| <b>Materialize</b>  | Components following <i>Material Design</i>                                          | Medium         |
| <b>Foundation</b>   | Configurable, more flexible components                                               | High           |

### 2.5. Best Practices

- Prefer external CSS; avoid `!important` and excessive *inline* styles
- Use a consistent naming convention for classes (for example, the *BEM* convention: `block__element--modifier`)
- Think about accessibility from the start: colour contrast, a `label` associated with each `input`, a logical keyboard-navigation order
- Always test the page's responsiveness, at different screen widths

### 2.6. Exercise: the Static Pages of the “World Capitals” Application

Applying what was presented in the previous sections, this section builds the visual pages of the application used throughout the practical LABs (see [Appendix D](#) and onward): an API serving information about countries and their capitals. At this stage, the pages are entirely static, in HTML/CSS/Bootstrap, with no JavaScript at all: the data shown is fixed directly in the HTML, and the forms are not yet connected to any server. That is the work reserved for the LABs, which will simply add `fetch()` to this already-finished code.

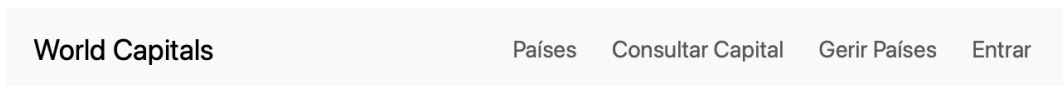
All the pages share the same navigation bar:

```

1 <nav class="navbar navbar-expand-md bg-light border-bottom">
2   <div class="container">
3     <a class="navbar-brand" href="index.html">World Capitals</a>
4     <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=
5       <span class="navbar-toggler-icon"></span>
6     </button>
7     <div id="nav1" class="collapse navbar-collapse">
8       <ul class="navbar-nav ms-auto gap-2">
9         <li class="nav-item"><a class="nav-link" href="index.html">Países</a></li>
10        <li class="nav-item"><a class="nav-link" href="capital.html">Consultar Capital</a>
11        <li class="nav-item"><a class="nav-link" href="países-editar.html">Gerir Países</a>
12        <li class="nav-item"><a class="nav-link" href="login.html">Entrar</a></li>
13      </ul>
14    </div>
15  </div>
16 </nav>

```

This code renders in the *browser* as shown in figure Figure 2.13:



**Figure 2.13.:** The navigation bar shared by the four pages of the “World Capitals” application, already styled by Bootstrap.

From this shared *navbar*, four pages are built:

- **Country listing** (`index.html`): a *Bootstrap* table, with a few countries fixed in the HTML as examples. In the LABs, this table becomes dynamically populated from the result of `/country`.
- **Authentication** (`login.html`): a form with the fields `user` and `pass`, with the same names expected by the `/login endpoint`.
- **Capital lookup** (`capital.html`): a simple form, with a single `countrycode` field.
- **Country management** (`países-editar.html`): three separate forms, one per operation, all pointing to the same `/countryEdit endpoint`, distinguished by the hidden `operation` field (see the note on this design in Chapter 7).

These four pages, linked to one another by the shared navigation bar, already form a small, complete, working Web site, even with no server-side logic at all: it is possible to navigate between them, fill in the forms, and submit them. What is still missing is a server to answer those requests, and that is what the material in the following chapters will provide.

The complete, self-contained code for the four pages, ready to copy into an editor and open directly in the *browser*, is gathered in Appendix A.

# 3. The HTTP Protocol

## 3.1. The Middleware Connecting Frontend and Backend

Chapter 1 identified three components in any Web application: *Frontend*, *Middleware* and *Backend*. Chapter 2 was devoted to the first of these, a set of HTML pages with forms, including the capital-lookup form (`capital.html`) and the authentication form (`login.html`). On their own, these forms do nothing: they need a protocol to carry the data the user fills in to a server, and to bring a response back. That protocol, the *Middleware* used in Web applications, is HTTP (*HyperText Transfer Protocol*). This chapter details how it works.

## 3.2. The TCP/IP Model

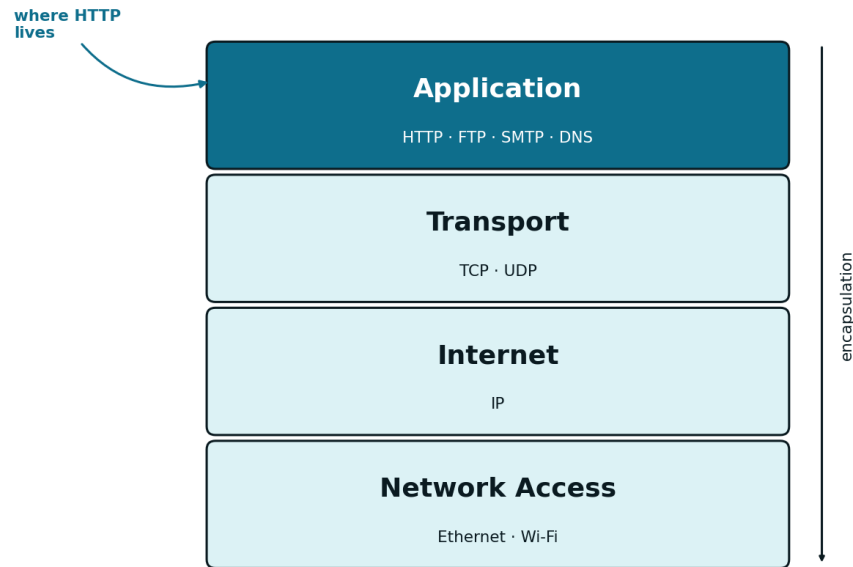
Before going into the details of HTTP itself, it is worth recalling where this protocol fits into the communications stack. The TCP/IP model organises that stack into four layers, each responsible for a different aspect of network communication:

| Layer          | Responsibility                                                  | Example protocols    |
|----------------|-----------------------------------------------------------------|----------------------|
| Application    | The format of the data exchanged between applications           | HTTP, FTP, SMTP, DNS |
| Transport      | Delivery of data between processes, reliable (TCP) or not (UDP) | TCP, UDP             |
| Internet       | Routing of data between networks, by IP address                 | IP                   |
| Network Access | Physical transmission of data on the local medium               | Ethernet, Wi-Fi      |

HTTP is, therefore, an Application-layer protocol: it is not concerned with how the data reaches its destination, only with defining the format of the request and the response. That delivery is TCP's responsibility, at the Transport layer, which guarantees that the data arrives



### 3. The HTTP Protocol



**Figure 3.1.:** The four-layer TCP/IP model. HTTP lives at the Application layer, and rests on TCP (Transport layer) to guarantee that data arrives reliably and in the correct order.

complete, error-free, and in the order it was sent — a guarantee that HTTP depends on, without ever implementing it directly.

## 3.3. HTTP

**HTTP** (*HyperText Transfer Protocol*) is a text-based protocol, based on the request-response model: the client sends a request, the server returns a response, and the connection between the two retains no memory of previous requests — each request is handled independently, that is, HTTP has no state (it is *stateless*). This section details the three aspects that define a request and a response: methods, the general message format, and status codes.

### 3.3.1. Methods

Every request from the client to the server is made in HTTP format and indicates, through a method, the operation intended for a given resource. The “resource” in the context of Web applications can be an `html` page, an image, a `PDF`, or any other item capable of being manipulated digitally:

| Method              | Typical semantics              |
|---------------------|--------------------------------|
| <code>GET</code>    | Retrieve a resource            |
| <code>POST</code>   | Create a resource              |
| <code>PUT</code>    | Update or replace a resource   |
| <code>DELETE</code> | Remove a resource              |
| <code>HEAD</code>   | Retrieve headers only, no body |

### 3. The HTTP Protocol

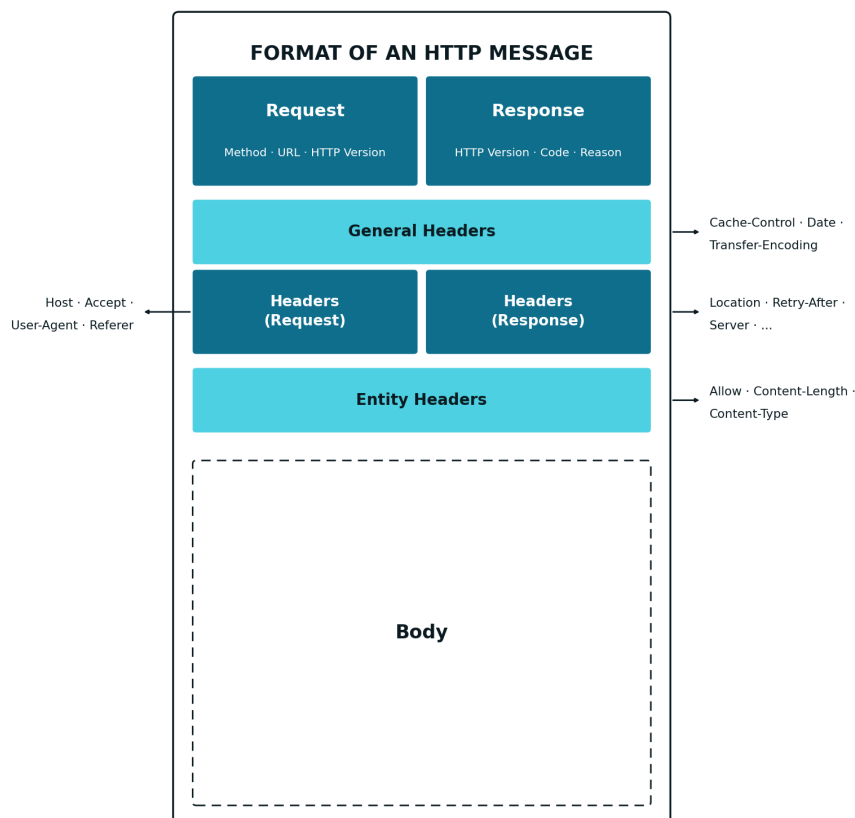
These five methods cover the vast majority of operations performed by a Web application; the first two (**GET** and **POST**) are by far the most common in Web pages, while **PUT** and **DELETE** become more important mainly in Web services.

#### 3.3.2. Message Format

When the application on the client's computer wants to communicate with the server, it activates the Application layer of the communications protocol stack, which formats the request according to the HTTP protocol. Both a request and the response that follows it share a similar general message structure:

1. A first line, differing between a request (method, *URL* and HTTP version) and a response (HTTP version, status code and reason)
2. A set of headers, carrying metadata about the message (for instance, the destination host, the content type, or its length)
3. An optional body, holding the actual data

A **GET** request typically has no body: all the data it carries goes in the *URL* itself, as parameters. A **POST** request carries its data in the message body, outside the *URL*.



**Figure 3.2.:** The general format of an HTTP message: the first line (different for a request and a response), followed by the various groups of headers, and finally the body.

##### 3.3.2.1. Example: From the Capital-Lookup Form to an HTTP Request

Recall the capital-lookup form built in [Chapter 2](#) (`capital.html`):

### 3. The HTTP Protocol

```
1 <form method="GET" action="/capital">
2   <label for="countrycode">Código do país</label>
3   <input type="text" id="countrycode" name="countrycode">
4   <button type="submit">Consultar</button>
5 </form>
```

This HTML code renders in the *browser* as shown in figure Figure 3.3:



**Figure 3.3.:** The capital-lookup form (`capital.html`), as displayed in the browser.

When submitted (by clicking the *Consultar* button) with the value PT in the `input`, the *browser* generates the following GET HTTP request:

```
GET /capital?countrycode=PT HTTP/1.1
Host: localhost:8080
```

Note that, because this is a GET method, the `countrycode` parameter is sent as part of the *URL* (after the `?`), and there is no body in the message. If the same form used `method="POST"`, the parameter would instead travel in the request body, and the *URL* would stay clean (`/capital`).

The server, upon processing this request, returns an HTTP response such as the following:

```
HTTP/1.1 200 OK
Content-Type: application/json
{"country": "Portugal", "capital": "Lisboa", "ccode": "PT"}
```

#### 3.3.2.2. Example: From the Authentication Form to a POST Request with a Body

Now recall the authentication form, also from [Chapter 2](#) (`login.html`), with the fields `user` and `pass`:

```
1 <form method="POST" action="/login">
2   <label for="user">Utilizador</label>
3   <input type="text" id="user" name="user">
4   <label for="pass">Palavra-passe</label>
5   <input type="password" id="pass" name="pass">
6   <button type="submit">Entrar</button>
7 </form>
```

This HTML code renders in the *browser* as shown in figure Figure 3.4:



**Figure 3.4.:** The authentication form (`login.html`), as displayed in the browser.

When submitted, this form generates the following HTTP request:

### 3. The HTTP Protocol

```
POST /login HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 21

user=javaee&pass=asw2026
```

Unlike the previous example, the parameters no longer go in the *URL*: they follow in the request body, after a blank line separating the headers from the body. This is why credentials and other sensitive data are typically sent via **POST**, never **GET**: a request body is not recorded in browser history or in the server's log files, unlike *URL* parameters.

Upon successfully validating the credentials, the server returns an HTTP response such as the following — in this case, a presentation-oriented application (see [Chapter 1](#)), which returns HTML code in the response body, ready to render in the *browser*. If the application were service-oriented, the response body would instead carry JSON rather than HTML:

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Bem-vindo, javaee</h1>
<p>Sessão iniciada com sucesso.</p>
```

#### 3.3.3. Status Codes

Every HTTP response includes a numeric status code, briefly indicating the outcome of the request. Codes are grouped into five categories, based on the leading digit:

| Range | Category      |
|-------|---------------|
| 1xx   | Informational |
| 2xx   | Success       |
| 3xx   | Redirection   |
| 4xx   | Client error  |
| 5xx   | Server error  |

Within these categories, a handful of codes come up particularly often when developing Web services:

| Code                      | Meaning                                                      |
|---------------------------|--------------------------------------------------------------|
| 200 OK                    | The request was processed successfully                       |
| 201 Created               | A new resource was successfully created                      |
| 400 Bad Request           | The request is malformed, or a required parameter is missing |
| 403 Forbidden             | The client lacks permission to access the resource           |
| 404 Not Found             | The requested resource does not exist                        |
| 500 Internal Server Error | An unhandled error occurred on the server side               |

Choosing the right status code for each situation is an essential part of designing a Web service: the code, by itself, conveys information the client can use to decide how to react, without needing to parse the response body.

# 4. Java EE and Servlet Fundamentals

## 4.1. The Jakarta EE Platform

With the Frontend built ([Chapter 2](#)) and the Middleware detailed ([Chapter 3](#)), what remains is the third piece identified in [Chapter 1](#): the Backend, which actually receives HTTP requests and answers them. That is the subject of this chapter and the ones that follow.

*Jakarta EE* (previously known as *Java EE*, until it was renamed in 2019) is a platform that provides a ready-to-use set of services and *APIs* to deal with the problems common to any Web application (see [Chapter 1](#)), freeing developers to focus on the application's business logic.

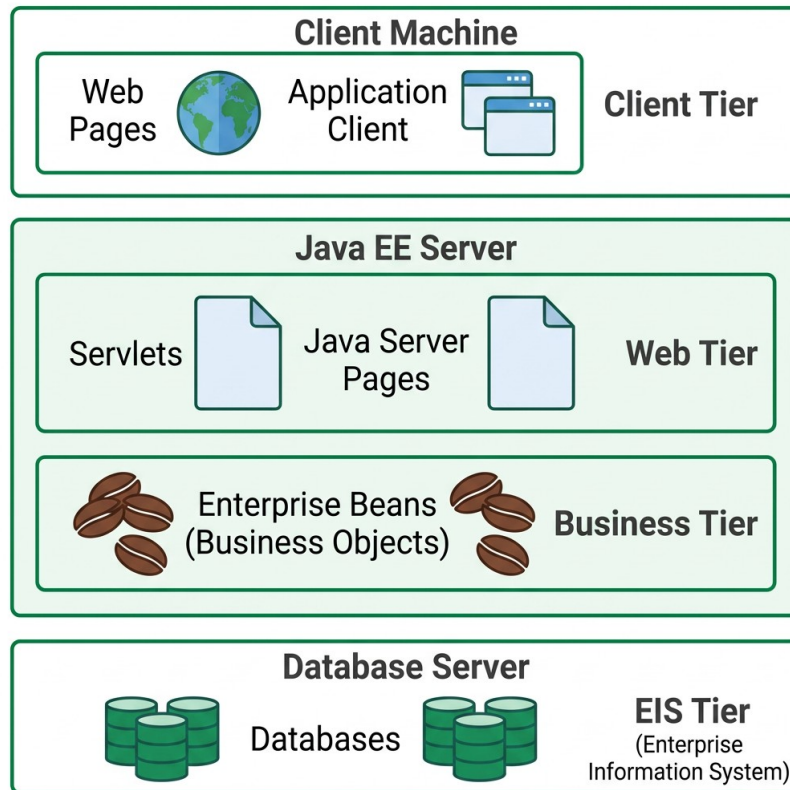
The platform rests on the Java language and the *Java Virtual Machine* (JVM), which means any component written for *Jakarta EE* benefits from the portability and automatic memory management already familiar from the language.

## 4.2. Layered Architecture

The *Jakarta EE* platform organises an application according to a model distributed across several layers (*tiers*), each with a well-defined role and typically installed on a different machine:

| Tier                                            | Where it resides         | Typical components                                              |
|-------------------------------------------------|--------------------------|-----------------------------------------------------------------|
| <i>Client Tier</i>                              | Client machine           | Web pages (browser), client applications                        |
| <i>Web Tier</i>                                 | <i>Jakarta EE</i> server | <i>Servlets</i> , <i>JavaServer Pages</i> , <i>Web Services</i> |
| <i>Business Tier</i>                            | <i>Jakarta EE</i> server | <i>Enterprise Beans</i> (business logic)                        |
| <i>EIS Tier (Enterprise Information System)</i> | Database server          | Databases, enterprise information systems                       |

This layering allows each component to evolve relatively independently of the others: changing the database does not require rewriting the business logic, and changing the client interface does not require changing the server.



**Figure 4.1.:** The layered architecture of Jakarta EE: the client communicates only with the Web Tier, which in turn relies on the Business Tier and, through it, on the database in the EIS Tier.

**i** Tomcat is not a complete Jakarta EE server

*Apache Tomcat*, used throughout this course, is a *Servlet Container*: it implements only the *Web Tier* (Servlets, JSP). It does not include support for *Enterprise Beans* nor for the *Business Tier* itself, unlike a complete *Jakarta EE* application server (for example, *WildFly*, *WebSphere* or *GlassFish*). In practice, this means that the business logic which, in the formal architecture, would belong to the *Business Tier*, is implemented here directly in plain Java classes, rather than in *Enterprise Beans*.

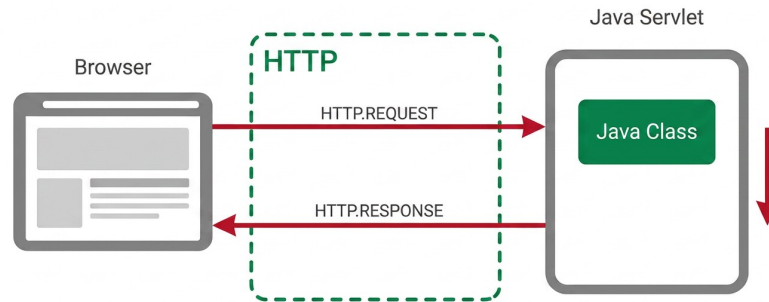
### 4.3. Why Servlets?

Implementing, from scratch, a Java program capable of answering HTTP requests would require solving, on its own, a set of problems common to any Web server: parsing the incoming HTTP message, managing several connections at once (each in its own *thread*), controlling the life cycle of the in-memory instances, and packaging all of this into a format that an application server knows how to install and start.

A *Servlet* is a Java class that runs inside a *Servlet Container* (for example, *Apache Tomcat*) and automatically receives all of this work already done:

- Parsing of the HTTP request
- *Thread* management, one per request
- Management of the *Servlet*'s own life cycle
- Its own deployment model, packaged in a `.war` file (*Web Application Archive*)

The *Servlet Container* corresponds, in the layered architecture presented above, to the *Web Tier*: it is there that *Servlets* reside, between the client and the business layer.



**Figure 4.2.:** From the client's point of view, a *Servlet* is just another destination for HTTP requests: it receives a request, returns a response.

### 4.4. The Life Cycle of a Servlet

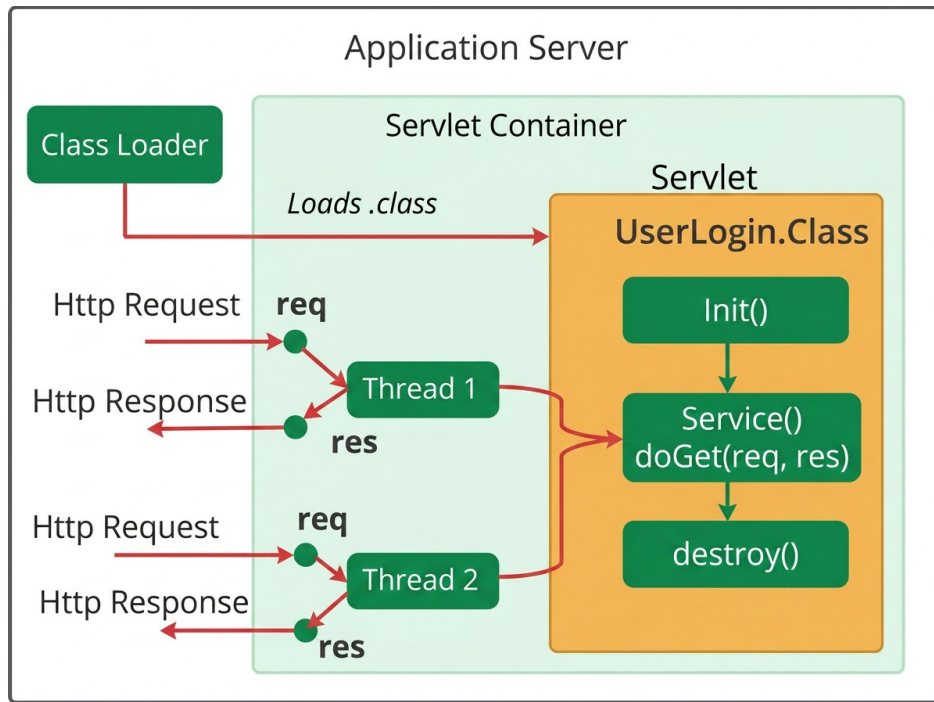
When a request is routed to a *Servlet*, the container always follows the same sequence (see Figure 4.3):

1. If no instance of that *Servlet* yet exists in memory, the container loads the class, creates an instance, and calls the `init()` method
2. For each request received, the container invokes the `service()` method, which dispatches to `doGet()`, `doPost()`, or another method depending on the HTTP verb used, passing it the `request` and `response` objects
3. If the container needs to remove the *Servlet* (for example, when shutting down the server), it invokes the `destroy()` method

An important detail follows from this: there is **a single instance** of each *Servlet* in memory, even if N clients are using the same service at once. Each request is handled by a different *thread*, but all of those *threads* share the same instance. In practice, this means:

#### ⚠ Beware of shared state

Never store data specific to one request in the *Servlet*'s instance attributes: since the instance is shared by all *threads*, one request can read or overwrite data belonging to another request in progress. Local variables, declared inside `doGet()` or `doPost()`, are safe, because each *thread* has its own copy.



**Figure 4.3.:** The container loads the Servlet's class only once; each incoming request is dispatched to its own thread, which invokes `service()` on that same shared instance.

## 4.5. From the HTTP Message to the Java Method

A *Servlet* is bound to a *URL* through the `@WebServlet` annotation, which maps a path to the class that will handle it:

```

1 @WebServlet("/country")
2 public class CountryServlet extends HttpServlet {
3     // ...
4 }

```

A client request to `http://localhost:8080/country` is thus routed to the `CountryServlet` class. The container automatically builds, from the elements of the incoming HTTP message (see Chapter 3 (cap03.qmd)), the **request** object; the complementary **response** object is what the Servlet uses to build the outgoing message. The elements of each message correspond to concrete methods of these two objects:

| HTTP element                                                                                    | Java access                             | When it is used                                                                |
|-------------------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------|
| Request headers                                                                                 | <code>request.getHeader(name)</code>    | Always, regardless of the type of request                                      |
| Parameters (GET query string, or <code>application/x-www-form-urlencoded</code> body of a POST) | <code>request.getParameter(name)</code> | Traditional HTML forms (see Chapter 2), in a presentation-oriented application |

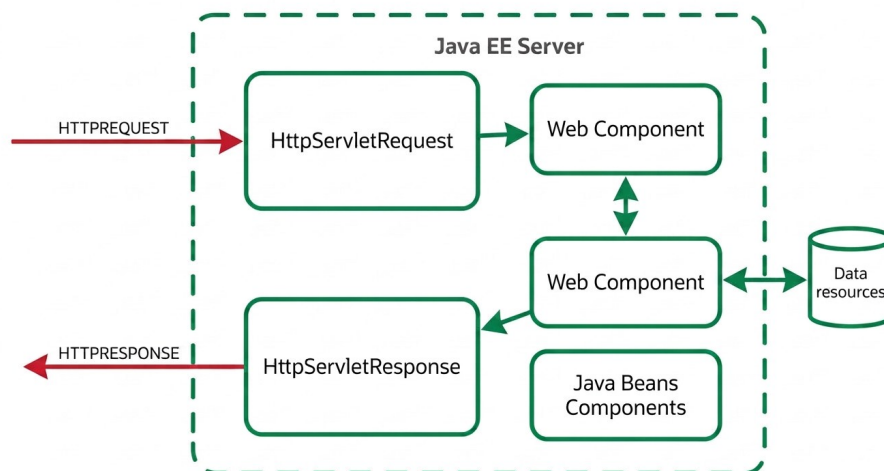


| HTTP element                      | Java access                                                                  | When it is used                                                                                                                                                                                                                               |
|-----------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Raw request body (typically JSON) | <code>request.getInputStream()</code>                                        | REST <i>Web services</i> (see <a href="#">Chapter 7</a> ): the body does not arrive in <code>application/x-www-form-urlencoded</code> format, so <code>getParameter()</code> does not parse it, and the JSON must be read and parsed manually |
| Writing the response              | <code>response.getWriter()</code> or <code>response.getOutputStream()</code> | Always, regardless of the type of response                                                                                                                                                                                                    |

```

1 protected void doGet(HttpServletRequest request, HttpServletResponse response) {
2     String countryCode = request.getParameter("countrycode");
3     // ...
4 }

```



**Figure 4.4.:** Inside the Jakarta EE server, the request arrives wrapped in an `HttpServletRequest`, is processed by the Web component (the Servlet), which may call another servlet, or query data resources, and the response goes out wrapped in an `HttpServletResponse`.

## 4.6. Content-Type and the Shape of the Response

Before writing anything to the response, it is essential to set its `Content-Type`: this header tells the client how to interpret the bytes that follow, whether as an HTML page or as [JSON](#) data.

For a presentation-oriented application:

```

1 response.setContentType("text/html");
2 PrintWriter out = response.getWriter();
3 out.println("<html><body>");
4 out.println("<h1>Hello João</h1>");
5 out.println("</body></html>");

```

For a service-oriented application:

```
1 response.setContentType("application/json");
2 PrintWriter out = response.getWriter();
3 out.print("{\"name\":\"João\",\"role\":\"admin\"}");
```

In the absence of a dedicated JSON-generation library, the most direct approach is to build the *String* by hand, taking care to escape internal quotes with `\`:

```
1 String jsonResponse = "{\"country\": \"Portugal\", \"capital\": \"Lisboa\", \"ccode\": \"\"";
2 out.print(jsonResponse);
3 out.flush();
```

To set the response’s status code (see [Chapter 3](#)), the `HttpServletResponse` class provides a constant for each code, avoiding “magic” numbers directly in the code:

| HTTP code | Java constant                                             |
|-----------|-----------------------------------------------------------|
| 200       | <code>HttpServletResponse.SC_OK</code>                    |
| 201       | <code>HttpServletResponse.SC_CREATED</code>               |
| 400       | <code>HttpServletResponse.SC_BAD_REQUEST</code>           |
| 403       | <code>HttpServletResponse.SC_FORBIDDEN</code>             |
| 404       | <code>HttpServletResponse.SC_NOT_FOUND</code>             |
| 500       | <code>HttpServletResponse.SC_INTERNAL_SERVER_ERROR</code> |

```
1 response.setStatus(HttpServletResponse.SC_NOT_FOUND);
```

## 4.7. A First Servlet: “World Capitals”

Applying what has been presented, a first *Servlet* can serve as a simple health check for the API:

```
1 @WebServlet("/status")
2 public class StatusServlet extends HttpServlet {
3     protected void doGet(HttpServletRequest request, HttpServletResponse response)
4         throws ServletException, IOException {
5         response.setContentType("text/plain");
6         PrintWriter out = response.getWriter();
7         out.print("OK");
8     }
9 }
```

A second one, already returning fixed data in JSON format, is the first step of the “World Capitals” application used throughout this course:

```

1  @WebServlet("/countrycapital")
2  public class CountryCapitalServlet extends HttpServlet {
3      protected void doGet(HttpServletRequest request, HttpServletResponse response)
4          throws ServletException, IOException {
5          response.setContentType("application/json");
6          PrintWriter out = response.getWriter();
7
8          String jsonResponse = "{\"country\": \"Portugal\", \"capital\": \"Lisboa\", \"cco
9          out.print(jsonResponse);
10         out.flush();
11     }
12 }

```

This *Servlet* always returns the same country, with the data fixed directly in the code — a deliberate simplification at this introductory stage. In a real application, this data would instead come from an object dedicated to data access, a *Data Access Object* (DAO).

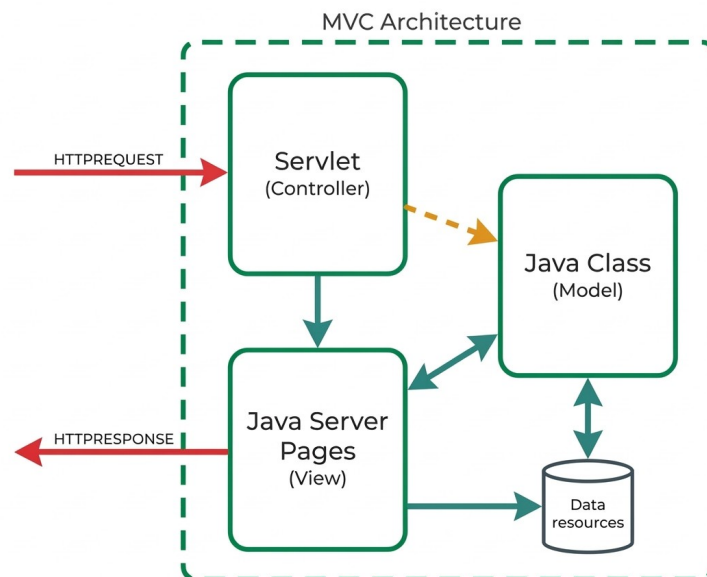
A DAO is not a formal *Jakarta EE* concept, nor does it require any special annotation or interface — it is simply a common convention: a plain Java class that isolates, in a single place, how data is obtained (for example, `findByCode(code)` or `validateUserCredentials(user, pass)`), hiding from the rest of the code whether, behind those methods, there is a fixed in-memory list, a database, or any other source.

## 4.8. Servlets and the MVC Pattern: the Missing View

This course focuses mainly on Web service applications, so the *Servlets* presented always return JSON: being service-oriented, they in fact have no View at all (see [Chapter 1](#)). This does not mean the application cannot feed a graphical view; in that case the MVC model is used, with the *Servlet* acting as **Controller** and feeding a Web page indirectly through JSON. That Web page is naturally built in HTML, but needs some script logic that extracts the necessary data from the JSON and places it in the right spots on the HTML page (see [Chapter 8](#)).

If the application is, from the ground up, presentation-oriented, however, the same *Servlet* that acts as **Controller** can generate the page's HTML (the **View**) directly, or it can delegate that responsibility to a dedicated page, using a technology known as JSP (*JavaServer Pages*), keeping MVC's separation of responsibilities, as shown in [Figure 4.5](#).

That dedicated page, the *JavaServer Page* (JSP), is covered in detail in [Chapter 5](#).



**Figure 4.5.:** In the classic realisation of MVC in Jakarta EE, the Servlet acts as Controller, a JavaServer Page as View, and a plain Java class as Model.

# 5. The Stateful Model: Sessions and JSP

## 5.1. HTTP Is Stateless: the Problem

The HTTP protocol is *stateless* (see [Chapter 1](#)): each request is independent, and the server does not, by itself, retain any memory of previous requests. At the protocol level, there is no notion whatsoever of a “user with an active session”.

This raises an immediate practical problem: if a user authenticates in one request, how does a following request, minutes later, know that this user is still authenticated? The answer relies on two complementary mechanisms:

- **Cookies:** a small piece of information stored on the **client** side, automatically resent by the *browser* on every following request
- **HttpSession:** an object stored on the **server** side, associated with an identifier that the client returns on every request

## 5.2. Cookies

A *cookie* is a key/value pair stored in the *browser*. When the server sends a *cookie*, it expects the client to return it on every subsequent HTTP request directed at the same domain, thereby implementing an identification mechanism layered on top of HTTP’s **stateless** nature.

Two types can be distinguished, depending on their persistence:

- **Session cookies:** kept by the client only for the duration of a browsing session (`setMaxAge(-1)`). While the session is not closed, the client keeps identifying itself through the *cookie*. From the moment the session is closed (by *logout* or by closing the *browser*), the *cookie* is destroyed.
- **Permanent cookies:** stored persistently, with a defined lifetime (`setMaxAge(60*60*24*30)` — the time is expressed in seconds, here for 30 days).

Creating a *cookie*:

```

1 Cookie cookie = new Cookie("userName", "Pedro");
2 cookie.setMaxAge(-1); // session cookie
3 response.addCookie(cookie); // the cookie is added to the response and sent to the client

```

Reading a *cookie* received in a request:

```

1 Cookie[] cookies = request.getCookies();
2 String user = null;
3
4 for (Cookie c : cookies) {
5     if (c.getName().equals("userName")) {
6         user = c.getValue();
7     }
8 }

```

### 5.3. Sessions (HttpSession)

Storing state on the server side requires a dedicated object: the `HttpSession`. A session is obtained, or created, from the request:

```

1 HttpSession session = request.getSession(); // creates it if it doesn't exist
2 HttpSession session = request.getSession(true); // equivalent, explicit
3 HttpSession session = request.getSession(false); // NEVER creates one; returns null if

```

Once obtained, the session stores and returns information through attributes, and can be explicitly terminated:

```

1 session.setAttribute("email", "foo@fool.com");
2 String email = (String) session.getAttribute("email");
3 session.invalidate(); // ends the session
4 session.setMaxInactiveInterval(1800); // inactivity time, in seconds

```

One piece is still missing: if HTTP has no memory of previous requests, how does the server know that two successive requests belong to the same session? The answer is, once again, *cookies*: by convention, the session identifier travels in a *cookie* named `JSESSIONID`, automatically created by the *Servlet Container* on the first call to `getSession()`.

`JSESSIONID=22562577CE80EF3B1DAAA0E48F2B6392`

When the client does not accept *cookies*, there is an alternative: encoding the session identifier directly in the *URL*.

`http://localhost:8080/CapitaisAPI/country;jsessionid=829F8836C240B51DD2F074259F242833`

It is good practice to always call `response.encodeURL(url)` when building a destination *URL*: if *cookies* are active, the method returns the *URL* unchanged; if they are not, it automatically appends the `jsessionid`.

## 5.4. Session-Based Authentication

Combining the two mechanisms above yields the classic session-based authentication pattern. After validating the credentials, the `LoginServlet` creates a session and stores the user's identifier in it:

```

1 @WebServlet("/login")
2 public class LoginServlet extends HttpServlet {
3     protected void doPost(HttpServletRequest request, HttpServletResponse response)
4         throws ServletException, IOException {
5         String user = request.getParameter("user");
6         String pass = request.getParameter("pass");
7
8         if (dao.validateUserCredentials(user, pass)) {
9             HttpSession session = request.getSession(true);
10            session.setAttribute("UserName", user);
11
12            response.setContentType("application/json");
13            response.getWriter().print("{\"message\":\"OK\",\"user\":\"" + user + "\"}");
14        } else {
15            response.setStatus(HttpServletResponse.SC_FORBIDDEN);
16            response.getWriter().print("{\"message\":\"FORBIDDEN\",\"error\":\"Invalid lo
17        }
18    }
19 }

```

Note that `dao` is an instance of a *Data Access Object* (DAO) — a dedicated data-access object, already mentioned in [Chapter 4](#) — responsible, in this case, for validating the credentials received: it typically queries a user database, and returns `true` or `false` depending on whether the `user/pass` pair is valid.

Any protected *endpoint* checks, at the start of its own `doGet()` or `doPost()`, whether a valid session with that attribute exists:

```

1 HttpSession session = request.getSession(false);
2 if (session == null || session.getAttribute("UserName") == null) {
3     response.setStatus(HttpServletResponse.SC_FORBIDDEN);
4     response.getWriter().print("{\"error\":\"Authentication required\"}");
5     return;
6 }
7 // authenticated request: continue normal processing

```

Logout simply amounts to ending the session:

```

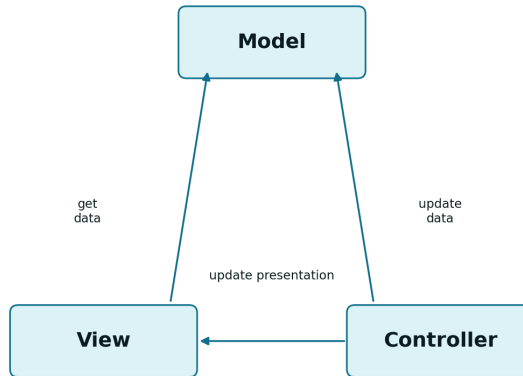
1 session.invalidate();

```

## 5.5. The MVC Pattern with *Servlet* and JSP

[Chapter 1](#) introduced the MVC pattern at a conceptual level, and [Chapter 4](#) showed how a *Servlet* acts as Controller. The pages built in [Chapter 2](#) already play the role of View, but

they are static: their content does not change depending on the request. This section shows an alternative way of building the View — this time generated dynamically on the server side, from the data the Controller hands it.



**Figure 5.1.:** The general MVC model, applicable to any interface (not just Web): the Controller updates the Model and the presentation; the View reads the Model to render itself.

In a presentation-oriented application, the *Servlet* receives the request and decides **what** to show, but delegates the actual **generation of HTML** to a *JavaServer Page* (JSP). The Model, in turn, is a plain Java class (for example, a *DAO*), independent of both.

The *Servlet* connects to the JSP through a *RequestDispatcher*: it places the data to be shown in a request attribute, and forwards the request itself to the page:

```

1 request.setAttribute("user", user);
2 request.getRequestDispatcher("/home.jsp").forward(request, response);

```

A JSP is a text document that mixes two kinds of content: static HTML and dynamic *script* elements. The first time it is requested, the *Servlet Container* automatically converts it into a *Servlet* class, which from then on behaves exactly like any other *Servlet*, in the role of Controller.

## 5.6. A Simple JSP Example

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2
3 <% String name = (String) request.getAttribute("userName"); %>
4
5 <!DOCTYPE html>
6 <html>
7 <head>
8   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
9   <title>A JSP Page</title>
10 </head>

```



```

11 <body>
12   <h1>Hello, <%= name %>!</h1>
13 </body>
14 </html>

```

The `<% ... %>` block (*scriptlet*) contains arbitrary Java code; the `<%= ... %>` block (expression) inserts the value of a Java expression directly into the generated HTML. It is exactly this `userName` attribute, placed in the request by the *Servlet* through `request.setAttribute(...)`, that the JSP retrieves here.

## 5.7. Lifecycle and Implicit Objects

A JSP's lifecycle follows the same model as the *Servlet* it is converted into (see [Chapter 4](#)), with two details of its own:

- The container automatically recompiles the JSP whenever the `.jsp` file changes
- Instead of `init()` and `destroy()`, the JSP offers the methods `jspInit()` and `jspDestroy()`

Another practical difference: inside a JSP, a set of objects is automatically available, with no prior declaration, called implicit objects:

| Implicit object       | Type                             |
|-----------------------|----------------------------------|
| <code>out</code>      | <code>JspWriter</code>           |
| <code>request</code>  | <code>HttpServletRequest</code>  |
| <code>response</code> | <code>HttpServletResponse</code> |
| <code>session</code>  | <code>HttpSession</code>         |
| <code>page</code>     | <code>Object</code>              |

Because of the implicit object `out`, a JSP never needs to explicitly obtain a `PrintWriter`: `out` is used directly.

## 5.8. Scopes: Sharing Information Within the Server

Just as *Servlets* can share information with other modules through attributes (see [Chapter 4](#)), those attributes always exist within a given *scope*. There are four kinds of *scope*:

- **Web context** (or *application context*): a single one per application, shared by all modules and all users
- **Session**: one per user, kept across several requests from that user
- **Request**: one per request, lost as soon as the response is sent
- **Page**: exclusive to JSP, not used in this course

All *scopes* follow the same interface: `setAttribute/getAttribute` methods, and `getAttributeNames()` to list everything stored there.

**Web context**, shared by the whole application:

```

1 ServletContext context = request.getServletContext();
2 context.setAttribute("dbHost", "localhost");
3 String dbHost = (String) context.getAttribute("dbHost");

```

**Session**, already introduced, shared only by the same user:

```

1 HttpSession session = request.getSession();
2 session.setAttribute("email", "foo@fool.com");
3 String email = (String) session.getAttribute("email");

```

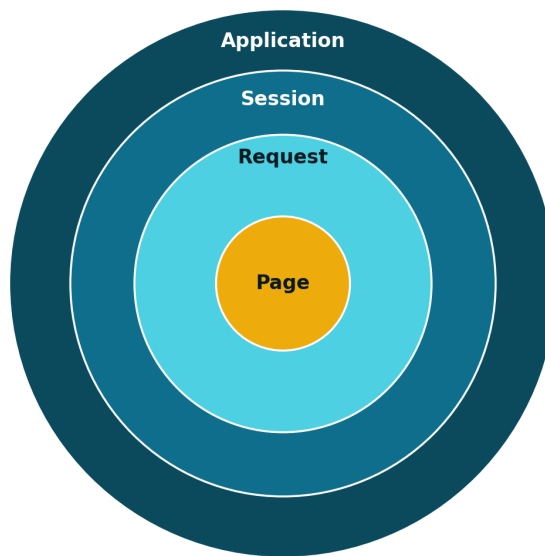
**Request**, the most restricted of all, shared only between modules processing the same request (for example, between a *Servlet* and the JSP it forwards to):

```

1 request.setAttribute("email", "foo@fool.com");
2 String email = (String) request.getAttribute("email");

```

The four *scopes* form an inclusion hierarchy, from the widest to the most restricted: *Application* contains *Session*, which contains *Request*, which contains *Page*. The wider the *scope*, the more widely shared the information, and the greater the responsibility to manage it correctly (for instance, to remove it when it is no longer needed).



larger scope → more sharing → more responsibility

**Figure 5.2.:** The four scopes, from the widest (Application) to the most restricted (Page): each contains the next, and a wider scope means more sharing, but also more responsibility in managing it.

#### **i** *Scopes* are not instantiated

Scope objects (`ServletContext`, `HttpSession`, `ServletRequest`) are automatically provided by the container. A new instance is never created with `new`: if one does not yet exist, the appropriate methods (`getServletContext()`, `getSession()`) create it or

return the existing one.

# 6. Security and Filters

## 6.1. The Problem: Security Repeated in Every *Servlet*

With what has been presented so far (see [Chapter 5](#)), authentication is solved: the `LoginServlet` validates the credentials and stores the user in the session, and each protected *endpoint* checks that session at the start of its `doGet()` or `doPost()`. This works, but it has a hidden cost: the authentication logic lives, duplicated, inside **every** protected *Servlet*.

This duplication brings several problems along with it:

- The same session check is repeated in every protected *endpoint*
- There is a real risk of inconsistency between these copies
- It is easy to forget the protection when creating a new *endpoint*

Put another way, this duplication violates the *DRY* principle (*Don't Repeat Yourself*): a responsibility should live in a single place, and repeated logic increases the maintenance cost whenever that logic needs to change. More concretely, this design violates the **Single Responsibility Principle**: each *Servlet* ends up with two distinct responsibilities, its own business logic, and the security logic imposed on it. This makes security policies hard to evolve, and the design poorly scalable, both in number of *endpoints* and in the complexity of the access rules.

## 6.2. Authentication vs. Authorisation

Before solving the problem, it is worth pinning down two often-confused concepts:

**Authentication** Answers “who is the user?”. It is the identity check, typically performed at *login* time.

```
1 if (username.equals("admin") && password.equals("1234")) {  
2     HttpSession session = request.getSession(true);  
3     session.setAttribute("UserName", username);  
4 }
```

**Authorisation** Answers “what is this user allowed to access?”. It is the permission check, performed once identity has already been confirmed.

```

1 if (session.getAttribute("UserRole").equals("ADMIN")) {
2     // access granted
3 }

```

Authentication always happens first: it makes no sense to check the permissions of someone whose identity has not yet been confirmed. This distinction is directly reflected in the HTTP status codes used in the response (see [Chapter 3](#)):

- **401 Unauthorized:** the request is not authenticated
- **403 Forbidden:** the request is authenticated, but lacks permission for the requested resource

### 6.3. What a Filter Is

The question left unanswered is where the security logic should live: inside each *Servlet*? Duplicated across every *endpoint*? Or centralised in a single component? Security is, by nature, a *cross-cutting concern*: it runs through the whole application, rather than belonging to a single module. This is exactly what the Filter exists for.

A [Filter](#) is a server-side component that intercepts HTTP requests:

- It runs **before** the destination *Servlet*
- It can also run **after** the *Servlet* (over the response)
- It can block, redirect, or modify the request or the response

Beyond authentication and authorisation, Filters are the natural mechanism for other cross-cutting concerns: activity *logging*, *CORS* (*Cross-Origin Resource Sharing*), or input validation. A Filter separates these concerns from the business logic, which stays exclusively in the *Servlet*.

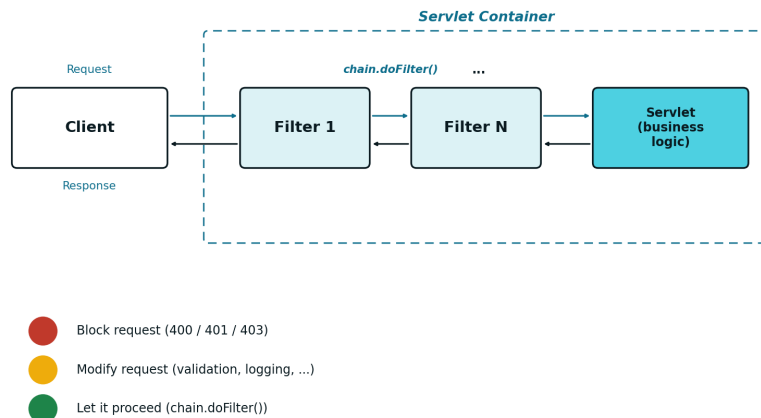
### 6.4. Where a Filter Sits in the Request

One or more Filters can be chained between the client and the *Servlet* that actually handles the request:

Client → Filter 1 → ... → Filter N → Servlet (business logic) → Response

Each Filter in the chain has, essentially, three possible behaviours:

- **Block the request:** for example, 400 (invalid data), 401 (invalid identity) or 403 (authenticated but without permission)
- **Modify the request:** data normalisation and validation, activity logging, among others
- **Let it proceed:** by calling `chain.doFilter(request, response)`, passing control to the next element in the chain



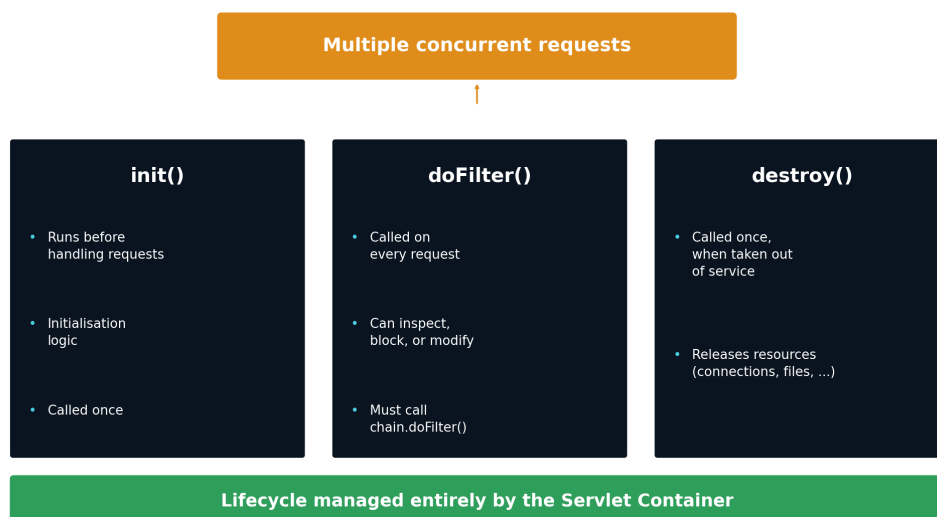
**Figure 6.1.:** A request passes through the Filter chain before reaching the Servlet; each Filter can block the request (red), modify it (yellow), or let it proceed (green, by calling `chain.doFilter()`).

## 6.5. A Filter's Lifecycle

A Filter's lifecycle follows the same model already presented for *Servlets* (see [Chapter 4](#)):

- `init()`: executed once, when the container puts the Filter into service
- `doFilter()`: executed on every request; it can inspect, block or modify the request and the response, and must call `chain.doFilter()` to continue the chain
- `destroy()`: executed once, when the Filter is taken out of service, to free resources

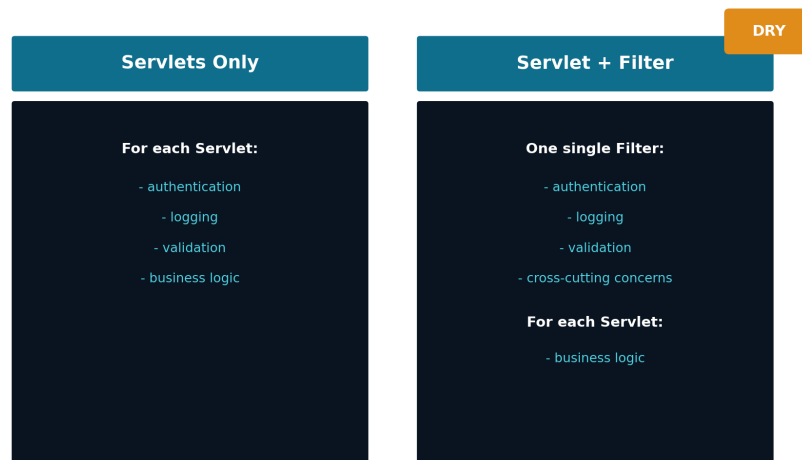
Just like a *Servlet*, there is **a single instance** of each Filter, shared by multiple concurrent requests: exactly the same care already mentioned in Chapter 4 applies here — avoid shared mutable state, and prefer local variables inside `doFilter()`.



**Figure 6.2.:** A Filter's lifecycle mirrors a Servlet's: `init()` and `destroy()` run once, `doFilter()` runs on every request, for multiple concurrent requests.

## 6.6. Servlet and Filter: Separation of Responsibilities

Centralising security in a Filter reorganises the responsibilities of the whole application:



*What happens if the authentication logic needs to change across 15 Servlets?*

**Figure 6.3.:** Without a Filter, each Servlet accumulates authentication, logging, validation and business logic. With a Filter, those cross-cutting concerns are concentrated in a single place, and each Servlet is left with only its own business logic.

This reorganisation brings concrete advantages:

- The authentication logic is written **once** (*DRY* principle)
- The security policy is applied consistently to every *endpoint*
- There is no longer any risk of “forgetting” the protection on a new *Servlet*
- The separation between cross-cutting concerns (Filter) and business logic (*Servlet*) becomes clear

## 6.7. Declaring and Implementing a Filter

A Filter is declared with the `@WebFilter` annotation, indicating the *URL* pattern it applies to:

```

1 @WebFilter("/*")
2 public class AuthenticationFilter implements Filter {
3     // ...
4 }
```

The pattern `/*` intercepts every request, with no exception. The class must implement `jakarta.servlet.Filter`, and the container discovers it automatically at *deployment* time. Just like *Servlets*, a Filter can also be declared equivalently in the `web.xml` file, although this practice is no longer much in use.

A typical `doFilter()`, protecting every request by requiring a valid session, has this form:

```

1 public void doFilter(ServletRequest request,
2                     ServletResponse response,
3                     FilterChain chain)
4     throws IOException, ServletException {
5
6     HttpServletRequest req = (HttpServletRequest) request;
7     HttpServletResponse res = (HttpServletResponse) response;
8
9     HttpSession session = req.getSession(false);
10
11     if (session == null || session.getAttribute("UserName") == null) {
12         res.sendError(HttpServletResponse.SC_UNAUTHORIZED); // 401
13         return;
14     }
15
16     chain.doFilter(request, response);
17 }

```

In practice, not every request should be protected: `/login` itself, and static resources such as `/css`, `/js` or `/images`, need to remain accessible without authentication — otherwise the user could never even reach the *login* form.

```

1 String path = req.getRequestURI();
2
3 boolean isLogin = path.endsWith("/login");
4 boolean isStatic = path.startsWith(req.getContextPath() + "/css")
5                     || path.startsWith(req.getContextPath() + "/js")
6                     || path.startsWith(req.getContextPath() + "/images");
7
8 if (isLogin || isStatic) {
9     chain.doFilter(request, response);
10    return;
11 }

```

## 6.8. Redirection vs. 401: an Architectural Difference

The correct way to react to an authentication failure depends on the kind of application:

- In a traditional, presentation-oriented Web application (see [Chapter 1](#)), the natural response is a **redirect** to the *login* page (code 302)
- In a REST API, whose contract is JSON (see [Chapters 7](#) and [8](#)), the correct response is to return **401**, with no redirection at all

The difference boils down to one sentence: in a traditional Web application, an authentication failure changes the page shown; in a REST API, it only changes the status code returned. It is the client, not the server, that decides what to do next with that 401.



## 6.9. Preparing for Stateless Security

The model presented in this chapter rests entirely on sessions: the server stores the authenticated user's state (`HttpSession`), and the Filter checks that session on every request. This model has one limitation: it requires the server to keep memory of each user, which becomes problematic as soon as there is more than one server instance (see [Chapter 8](#)).

The alternative is token-based authentication: instead of the server storing any session at all, each request itself carries, on every call, all the information needed to authenticate. This shift, from a stateful model to a stateless model, is the subject of the following chapters.

# 7.

## REST Web Services and Stateless Architecture (Part 1)

### 7.1. The Problem: Applications That Need to Communicate

Up to this point, the server has always been thought of from the perspective of a single kind of client: a *browser*, requesting HTML pages for a human user to read. But a server rarely serves only that kind of client. A mobile application, another server, an integration script, or the JavaScript running in the *browser* itself (see [Chapter 8](#)) may also need to access the same information — without any of them needing, or wanting, to receive a ready-to-display HTML page.

HTML is a format designed for presentation: it mixes data with the form, or semantic structure, in which that data should be shown. A program that only needs the data (for example, the price of a product, or a country’s capital) would have to “take apart” the HTML to get to it — fragile, and dependent on presentation details that have nothing to do with the information itself. What is needed is a way of exposing the same data so that any program, not just a *browser* with a human looking at it, can consume it directly.

### 7.2. What a Web Service Is

A *Web Service* is a software component accessible over the network — typically via HTTP — that exposes functionality or data in a format designed to be consumed by other programs, not read directly by a human. Unlike a Web page, a *Web Service* does not generate presentation: it returns structured data with no semantic dressing, and it is the client that decides what to do with it.

This decoupling is what allows the same server to answer, with the same logic, requests coming from completely different sources: a mobile application, a JavaScript *front-office*, or another server — all receive the same data, in the same structure, and each decides how to present it (or whether to present it at all).

### 7.3. SOAP and REST: Two Approaches

Historically, there have been two main approaches to building *Web Services*:

**SOAP (*Simple Object Access Protocol*)** A formal, XML-based protocol, with a rigid message structure (the SOAP *envelope*) and a contract described separately, in a *WSDL* document. It is verbose and requires specific tooling to generate and consume that contract, but it offers strong formal guarantees — used mostly in legacy enterprise systems, with strict transaction and security requirements.

**REST (*Representational State Transfer*)** An architectural style, not a formal protocol, that uses HTTP directly as both transport and contract: the verbs, status codes and headers already seen in [Chapters 3](#) and [4](#) are, themselves, the vocabulary of the API. It is lighter, simpler to test, and is the dominant approach today for Web and mobile applications — the only one covered in detail in this course.

### 7.4. REST: Fielding’s Principles

The term REST was coined by Roy Fielding, in 2000, to describe the architectural style that was already, in practice, implicit in the design of the Web itself. A REST service follows a set of principles:

- **Client-server:** responsibilities are separated; the client does not need to know how the data is stored, and the server does not need to know how it is presented,
- **Stateless:** each request contains all the information needed to be processed, without depending on memory of previous requests on the server (see [Section 7.9](#)),
- **Uniform interface:** resources are always manipulated the same way, through HTTP’s own verbs and status codes, without inventing a bespoke vocabulary per API,
- **Resources and representations:** everything the API exposes is modelled as a resource, identified by a *URL*, with one or more possible representations (typically JSON),
- **Cacheable:** a response can indicate whether or not it may be cached, to avoid unnecessary repeated requests.

Not every API that calls itself “REST” strictly follows all of these principles — in practice, “REST” has become a generic term for “resource-oriented API over HTTP, typically with JSON”. That is the sense used throughout this course.

### 7.5. Resource-Oriented Architecture

REST’s most visible principle is thinking of the API around **resources**, not actions. A resource is anything that can be identified and manipulated — a country, a user, an order — and it is identified by a *URL*, never by a function name.

A design that does not follow this principle is closer to an *RPC* (*Remote Procedure Call*) style, where the *URL* names an operation:

```
GET /getCountry?code=PT
GET /deleteCountry?code=PT
```

A resource-oriented design uses the *URL* itself to identify the resource, leaving the HTTP verb (see [Chapter 3](#)) to indicate the operation to perform on it:

GET /country/PT → get country PT

DELETE /country/PT → remove country PT

The advantage is not just aesthetic: it makes the API predictable. Once the `/country/{code}` pattern is known, it is immediately clear how to access, update or remove any country, without needing to memorise a different function name for each operation.

## 7.6. HTTP Methods as the API's Contract

In a REST API, HTTP methods stop being just “the way the request arrives” (see [Chapter 3](#)) and become the API's own contract: the meaning of each verb, applied to a resource, is always the same.

| Method | Operation                      | Idempotent? |
|--------|--------------------------------|-------------|
| GET    | Read the resource              | Yes         |
| POST   | Create a new resource          | No          |
| PUT    | Update or replace the resource | Yes         |
| DELETE | Remove the resource            | Yes         |

A method is said to be **idempotent** when repeating it several times has exactly the same effect as executing it once. DELETE /country/PT removes country PT; repeating the same request does not remove “anything more”, the final result is identical. POST /country, on the other hand, creates a new resource on every call, so it is not idempotent: repeating the request creates several resources, not just one. This property matters especially on unreliable networks: a client unsure whether a PUT or DELETE request reached its destination can safely repeat it; the same cannot be done with a POST, on pain of duplicating the created resource.

## 7.7. JSON as a Resource's Representation

A resource can have more than one possible representation — the same country could, in theory, be returned in JSON, in XML, or in any other format, depending on what the client requests. In practice, and throughout this course, the representation used is always **JSON** (the format traditionally associated with SOAP is XML, more verbose and less straightforward to generate and parse in JavaScript).

```
1 {"code": "PT", "name": "Portugal", "capital": "Lisboa"}
```

The `Content-Type: application/json` header (see [Chapter 4](#)) is what informs the client that this is the representation chosen for the response.

## 7.8. From a Web Application to a REST Service

The infrastructure behind a REST service is nothing new: it is still a *Servlet*, mapped to a *URL* through `@WebServlet` (see [Chapter 4](#)), receiving an HTTP request and returning a response. What changes is **what** that response contains, and **who** it is designed for.

In a presentation-oriented application (see [Chapters 1](#) and [5](#)), the Servlet returns HTML, meant for a human to read directly in the *browser*. In a REST API, the same Servlet returns JSON, meant to be interpreted by code — whether that code is another server, a mobile application, or the *browser*’s own JavaScript (see [Chapter 8](#)). The View, in the classic MVC sense (see [Chapter 1](#)), stops existing on the server side: it is the client, whatever it may be, that decides how (or whether) to present the data received.

## 7.9. Stateful vs. Stateless

[Chapter 5](#) presented a *stateful* model: the server keeps, in the `HttpSession`, information about each authenticated user, and every subsequent request is interpreted in light of that memory. A **stateless** service follows the opposite principle: the server keeps no memory at all between requests, and each request must carry, within itself, all the information needed to be processed — including, if required, proof that whoever is making it is authenticated.

This does not mean the application as a whole stops having persistent data — the database keeps storing countries, users, orders. It only means that the **application server** itself keeps no session information about who is making each request, from one request to the next.

## 7.10. Why the Absence of State Matters

The limitation of the stateful model only becomes visible once there is more than one server instance answering requests — a common situation in any application with a real need to scale. If a user authenticates on instance A, and a following request is routed to instance B, that second instance has no corresponding session in its memory: the user appears unauthenticated, despite having just logged in.

A stateless service does not suffer from this problem: since no instance keeps memory of anyone, any one of them can answer any request, indifferently. This property is what makes it possible to distribute requests across several server instances with no coordination between them at all — a topic picked up in detail in [Chapter 8](#).

### **i** POST + operation as an alternative to “pure” HTTP verbs

In practice, it is common to find APIs that do not follow the “pure” semantics of HTTP verbs presented above (POST to create, PUT to update, DELETE to remove). A pragmatic alternative, used especially in contexts with technical constraints — for example, when only GET/POST requests can be issued, as from a simple HTML form, with no JavaScript involved (see [Chapter 2](#)) — is to concentrate write operations in a single *endpoint*, always received via POST, distinguishing the intended operation through an additional parameter, such as `operation` (`insert`, `update`, `delete`).

This solution works, but moves away from more conventional REST design: a single *endpoint* no longer maps directly to a resource and a verb, drifting instead towards an *RPC* style over HTTP. In an API designed to be consumed by JavaScript — which can issue PUT and DELETE requests with no limitation at all (see [Chapter 8](#)) — the closer-to-REST alternative is to rewrite that *endpoint* as several, one per HTTP verb (for example, PUT `/resource/{id}` to update and DELETE `/resource/{id}` to remove).

# 8. REST Web Services (Part 2) and the JavaScript Client

## 8.1. Session Limitations in Distributed Systems

[Chapter 7](#) ended by pointing at the problem: an authentication model based on `HttpSession` (see [Chapter 5](#)) requires the server to keep, in memory, the state of each authenticated user. As soon as there is more than one server instance, that memory is no longer automatically shared between them.

There are known workarounds for this problem — *sticky sessions* (always routing the same client to the same instance) or replicating sessions across instances — but both come at a cost: the first limits how requests can be distributed, and the second requires extra infrastructure just to keep instances synchronised. Neither solves the problem at its root; both are ways of working around a dependency on state that, for a REST API (see [Chapter 7](#)), should not exist in the first place.

## 8.2. Stateless Authentication: the Authorization Header

The stateless alternative reverses the responsibility: instead of the server keeping track of who is authenticated, it is the **client itself** that keeps proof of its authentication — a *token* — and resends it with every request, typically in the HTTP **Authorization** header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJqYXZhZWUifQ.dGhpc19pc19hX2Zha2Vfc2lnbmF
```

The word **Bearer** indicates the type of credential that follows (literally, “bearer”: whoever presents this *token* is treated as its rightful owner). Upon receiving the request, the server does not need to consult any memory: it only needs to validate the *token* itself — what that is, and how that validation works, is the subject of the next section.

## 8.3. JWT: a Stateless Token

A *JSON Web Token* (JWT) is a standardised *token* format, designed to carry information compactly and verifiably, without the server needing to keep anything about it. A JWT always has three parts, separated by dots:

<header>.<payload>.<signature>

**Header** Metadata about the *token* itself, typically the signing algorithm used — for example, `{"alg": "HS256", "typ": "JWT"}`

**Payload** The data the *token* carries (the *claims*) — for example, `{"sub": "javaee", "role": "user", "exp": 18}` identifying the user, their role, and the expiry date

**Signature** The result of cryptographically signing the *header* and the *payload* with a secret key, known only to the server

Each of these three parts is encoded in *Base64* and concatenated with dots, producing the long, unreadable *string* characteristic of a JWT (like the one in the **Authorization** header example above). The *header* and the *payload* are only encoded, not encrypted — anyone can read them, with no key needed. What guarantees that no one can **forge** a valid *token* is the signature: only whoever knows the server's secret key can generate it correctly, and any change to the *header* or the *payload*, however small, invalidates the signature. Upon receiving a request, the server recomputes the signature from the received *header* and *payload*, and compares it with the one carried in the *token*: if they match, the content can be trusted, without the server having had to keep, itself, absolutely anything.

## 8.4. The Lifecycle of a *Token*

A *token* does not replace the moment of authentication itself — it only replaces the way the server remembers it.

**Issuance** The client authenticates exactly as in [Chapter 5](#), sending `user/pass` to a *login endpoint*. The server validates the credentials (for example, through a DAO, see [Chapter 4](#)) but, instead of creating an `HttpSession`, generates a new JWT, signs it with its secret key, and returns it to the client in the response body

**Storage and resending** Unlike `JSESSIONID` (see [Chapter 5](#)), which the *browser* resends automatically through a *cookie*, the client is responsible for storing the received *token* and for explicitly resending it in the **Authorization** header of every subsequent request

**Validation** Each request is validated completely independently of previous ones: the server recomputes the signature, without consulting any session at all (see [Section 8.5](#))

**Expiry** The `exp claim` in the *payload* defines how long the *token* remains valid. Once that moment has passed, the server rejects it (401) even if the signature is still correct, forcing a new *login*. This limits the time window in which an intercepted *token* could be misused — unlike an `HttpSession`, which only ends through explicit *logout* or prolonged inactivity (see [Chapter 5](#)), a JWT always expires, even if it is still being used

The following table summarises the difference between the two authentication models seen so far in this course:

|                            | Session ( <code>HttpSession</code> )       | JWT                                               |
|----------------------------|--------------------------------------------|---------------------------------------------------|
| Where the state lives      | On the server, in memory                   | In the <i>token</i> itself, on the client side    |
| What the client keeps      | An identifier ( <code>JSESSIONID</code> )  | The full <i>token</i> , already carrying the data |
| Validation on each request | A lookup in the server's memory            | Signature verification, with no lookup at all     |
| Multiple server instances  | Need to share the session with one another | Each validates independently                      |

In practice, a Filter (see [Chapter 6](#)) remains the natural place for this validation to happen. Recall the session-based authentication Filter already presented in that chapter:

```

1 public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
2     throws IOException, ServletException {
3
4     HttpServletRequest req = (HttpServletRequest) request;
5     HttpServletResponse res = (HttpServletResponse) response;
6
7     HttpSession session = req.getSession(false);
8
9     if (session == null || session.getAttribute("UserName") == null) {
10         res.sendError(HttpServletResponse.SC_UNAUTHORIZED); // 401
11         return;
12     }
13
14     chain.doFilter(request, response);
15 }

```

The equivalent, *token*-based version has exactly the same structure — only the check itself changes, from an `HttpSession` lookup to validating the *token*'s signature and validity. That validation is rarely written from scratch: it is sensitive code, involving cryptography, and it pays off to leave it to a well-tested, actively maintained library, rather than reimplementing it. [JJWT](#) is one of the most widely used in Java, and one of the more security-conscious in the ecosystem (for example, it requires the expected algorithm to be stated explicitly, to prevent a common attack in which a *token* with no signature at all would be accepted as valid):

```

1 public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
2     throws IOException, ServletException {
3
4     HttpServletRequest req = (HttpServletRequest) request;
5     HttpServletResponse res = (HttpServletResponse) response;
6
7     String authHeader = req.getHeader("Authorization");
8
9     if (authHeader == null || !authHeader.startsWith("Bearer ")) {
10         res.sendError(HttpServletResponse.SC_UNAUTHORIZED); // 401
11         return;
12     }
13
14     String token = authHeader.substring("Bearer ".length());
15
16     try {
17         Claims claims = Jwts.parser()
18             .verifyWith(SECRET_KEY)           // the server's secret key
19             .build()
20             .parseSignedClaims(token)
21             .getPayload();
22
23         req.setAttribute("UserName", claims.getSubject());
24     } catch (JwtException e) {

```



```

25     res.sendError(HttpServletResponse.SC_UNAUTHORIZED); // invalid signature or expir
26     return;
27 }
28
29 chain.doFilter(request, response);
30 }

```

The call to `parseSignedClaims(token)` internally does everything the validation requires: it decodes the *header* and the *payload*, recomputes the signature with `SECRET_KEY` and compares it with the one received, and checks the *exp claim* — throwing a `JwtException` (or one of its subclasses, such as `ExpiredJwtException` for an expired *token*, or `SignatureException` for an invalid signature) should any of these checks fail. Note the structural similarity to the session-based Filter: in both cases, a failure condition ends the request with 401, and the chain only continues with `chain.doFilter()` if authentication is valid.

## 8.5. Horizontal Scalability

Scaling an application means increasing its capacity to answer more requests. There are, typically, two ways of doing this:

**Vertical scalability** Increasing the resources (CPU, memory) of the existing machine — has a physical limit, and does not solve the problem of a single point of failure

**Horizontal scalability** Adding more server instances, running in parallel, distributing requests between them — typically through a *load balancer*

Horizontal scalability only works without complications when any instance can answer any request, indifferently — exactly the property a stateless service offers. With JWT-based authentication, no instance needs to know anything about any other: each validates the received *token* completely independently, using the same secret key shared by all of them. It is this independence between instances that makes stateless architecture the foundation of any system designed to scale.

## 8.6. Distributed MVC: the Pattern Splits Across Server and Client

In a presentation-oriented application, the MVC pattern (see [Chapter 1](#)) lives entirely on the server side: one component acts as Controller, and another, responsible for generating HTML, as View. In a REST API, that organisation stops making sense: the server no longer generates HTML, and the View, as defined in classic MVC, disappears from the server side.

On the server side, only the Model (data access) and the Controller (the *endpoint* that receives the HTTP request and returns JSON) remain. The role of View moves entirely to the client: it is the [JavaScript](#) code, running in the *browser*, that interprets the received JSON and manipulates the *DOM* to present it to whoever is using the application.

This split can be read as a second application of the same pattern, now on the client side: the JavaScript that handles the response and updates the page itself acts as both Controller (deciding what to do with the received response) and View (generating the HTML shown); the JSON returned by the API is the Model, kept on the server and synchronised on every `fetch()` request.

This is why a JavaScript *front-office* connected to a REST API is not an alternative to MVC, but rather the same pattern distributed across two machines instead of one.

## 8.7. Consuming a REST API in JavaScript: `fetch()` and JSON

The [Fetch API](#) is the standard way for a *browser* to issue HTTP requests from JavaScript, without reloading the page. A simple, unauthenticated request looks like this:

```
1 fetch("/country/PT")
2   .then(response => response.json())
3   .then(data => {
4     console.log(data.capital);
5   });
```

`fetch()` returns a *promise* — a value that is not yet available, but will be, later, once the request finishes. The first `.then()` receives the raw HTTP response, and converts its body, as JSON text, into a plain JavaScript object through `response.json()` — this step, in turn, also returns a *promise*, so it is chained with a second `.then()`, with the data now ready to use.

For an authenticated request (see [Section 8.2](#) and [Section 8.3](#)), the *token* travels in a header, just like any other authenticated HTTP request:

```
1 fetch("/country", {
2   headers: { "Authorization": "Bearer " + token }
3 })
4   .then(response => response.json())
5   .then(data => {
6     // ...
7   });
```

A POST request, creating a resource, carries its body already in JSON, explicitly stating that format in the `Content-Type` header (see [Chapter 4](#)):

```
1 fetch("/country", {
2   method: "POST",
3   headers: { "Content-Type": "application/json" },
4   body: JSON.stringify({ code: "PT", name: "Portugal", capital: "Lisboa" })
5 })
6   .then(response => response.json())
7   .then(data => {
8     // ...
9   });
```

## 8.8. The DOM as the *Browser's* “View”

The *DOM* (*Document Object Model*) is the in-memory representation of the HTML page the *browser* is displaying — a tree of objects, one per page element, that JavaScript can

read and change directly. It is through the DOM that JavaScript plays the role of View (see Section 8.6): it does not generate HTML as text, the way a JSP does (see Chapter 5), but instead locates elements already present on the page and updates their content.

```

1 fetch("/country/PT")
2   .then(response => response.json())
3   .then(data => {
4     document.querySelector("#capital").textContent = data.capital;
5   });

```

`document.querySelector(selector)` locates the first page element matching the given selector (here, the element with `id="capital"`), returning a reference to that DOM element; `textContent` is the property that sets the text shown inside it. The same principle applies to a list of results: for each item returned by the API, a new HTML element is created and inserted into the page.

```

1 fetch("/country")
2   .then(response => response.json())
3   .then(countries => {
4     const table = document.querySelector("#tabela-paises");
5     countries.forEach(country => {
6       const row = document.createElement("tr");
7       row.innerHTML = `<td>${country.code}</td><td>${country.name}</td><td>${country.capi
8     table.appendChild(row);
9   });
10  });

```

This pattern — requesting data from a REST API with `fetch()`, and using the DOM to display it — is the foundation of any JavaScript *front-office* connected to a REST service, regardless of the specific application or data involved.

# A. Complete Code: Static “World Capitals” Pages

## A.1. Introduction

This appendix gathers, into complete, self-contained files, the code presented piece by piece in [Chapter 2](#). Each block below is an independent `.html` file, ready to copy into an editor (VSCode or WebStorm) and open directly in the *browser*, with no installation or server configuration required: *Bootstrap* is loaded via *CDN*, and the data shown is fixed directly in the HTML.

This code serves two purposes:

- In class, as an immediate demonstration: copy it, open it in the *browser*, and have a small, working Web site to navigate right away
- In the practical LABs (see [Appendix D](#) onward), as *boilerplate*: the starting point to which the JavaScript needed to consume the REST API is added, without rewriting HTML from scratch

The four files share the same navigation bar and must be saved in the same folder, so that the links between them work: `index.html`, `login.html`, `capital.html` and `países-editor.html`.

**i** A note on the visible text in this appendix’s code

The on-screen text inside the four HTML files below (navigation labels, headings, form fields, button captions) is left in Portuguese, exactly as in the Portuguese edition. These are the same literal files, reused verbatim by the practical LABs later in the book ([Appendix D](#) onward), and by the screenshots in [Chapter 2](#) — keeping the text identical keeps code and screenshots consistent across both editions.

## A.2. Initial Project Structure

Before copying the code, the folder structure that will accompany the project throughout the course is created — from the static files in this appendix to the JavaScript *front-office* the practical LABs will add to them:

### A. Complete Code: Static “World Capitals” Pages

1. Create a project folder (for example, `world-capitals/`)
2. Inside it, place the four `.html` files from this appendix
3. Create three subfolders, still empty: `css/`, `js/` and `assets/`

```
world-capitals/  
  index.html  
  login.html  
  capital.html  
  paises-editor.html  
  css/  
  js/  
  assets/
```

- `css/`: holds style sheets linked to the `html` pages. It can also hold adjustments to the *Bootstrap* loaded via *CDN*, for instance a `custom.css` with visual tweaks specific to this application. A single style sheet can be used for all the pages, or each page can have its own.
- `js/`: for the *JavaScript* files. It stays empty for now; this is where, in the practical LABs, the `fetch()` code that connects these pages to the REST API will go
- `assets/`: images, icons, or other static files the application may need

This structure is not required by the *browser*: a single, isolated `.html` file, opened directly, already works, as can be confirmed in the following sections. It is, however, the organisation that any real Web project ends up adopting, and it is worth keeping in mind from the start.

### A.3. `index.html`: Country Listing

The following page has a navigation bar at the top and a table of countries. *Bootstrap* is loaded via *CDN*.

```
1  <!DOCTYPE html>  
2  <html lang="pt">  
3  <head>  
4    <meta charset="UTF-8">  
5    <meta name="viewport" content="width=device-width, initial-scale=1">  
6    <title>World Capitals</title>  
7    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" re  
8  </head>  
9  <body>  
10   <nav class="navbar navbar-expand-md bg-light border-bottom">  
11     <div class="container">  
12       <a class="navbar-brand" href="index.html">World Capitals</a>  
13       <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-targ  
14         <span class="navbar-toggler-icon"></span>  
15     </button>  
16     <div id="nav1" class="collapse navbar-collapse">  
17       <ul class="navbar-nav ms-auto gap-2">  
18         <li class="nav-item"><a class="nav-link active" href="index.html">Países</a></li>  
19         <li class="nav-item"><a class="nav-link" href="capital.html">Consultar Capital</li>  
20         <li class="nav-item"><a class="nav-link" href="paises-editor.html">Gerir Países
```

#### A. Complete Code: Static “World Capitals” Pages

```
21         <li class="nav-item"><a class="nav-link" href="login.html">Entrar</a></li>
22     </ul>
23 </div>
24 </div>
25 </nav>
26
27 <main class="container my-4">
28     <h1 class="display-6">Países</h1>
29     <table class="table table-striped">
30         <thead>
31             <tr><th>Código</th><th>País</th><th>Capital</th></tr>
32         </thead>
33         <tbody>
34             <tr><td>PT</td><td>Portugal</td><td>Lisboa</td></tr>
35             <tr><td>FR</td><td>França</td><td>Paris</td></tr>
36             <tr><td>EN</td><td>Reino Unido</td><td>Londres</td></tr>
37         </tbody>
38     </table>
39 </main>
40
41 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
42 </body>
43 </html>
```

#### A.4. login.html: Authentication

```
1 <!DOCTYPE html>
2 <html lang="pt">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>World Capitals - Entrar</title>
7     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" re
8 </head>
9 <body>
10     <nav class="navbar navbar-expand-md bg-light border-bottom">
11         <div class="container">
12             <a class="navbar-brand" href="index.html">World Capitals</a>
13             <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-targ
14                 <span class="navbar-toggler-icon"></span>
15             </button>
16             <div id="nav1" class="collapse navbar-collapse">
17                 <ul class="navbar-nav ms-auto gap-2">
18                     <li class="nav-item"><a class="nav-link" href="index.html">Países</a></li>
19                     <li class="nav-item"><a class="nav-link" href="capital.html">Consultar Capital<
20                     <li class="nav-item"><a class="nav-link" href="paises-editar.html">Gerir Países
21                     <li class="nav-item"><a class="nav-link active" href="login.html">Entrar</a></li>
22                 </ul>
```

## A. Complete Code: Static “World Capitals” Pages

```
23     </div>
24   </div>
25 </nav>
26
27 <main class="container my-4">
28   <h1 class="display-6">Entrar</h1>
29   <form class="row g-3 col-lg-6" method="POST" action="/login">
30     <div class="col-12">
31       <label for="user" class="form-label">Utilizador</label>
32       <input id="user" name="user" type="text" class="form-control">
33     </div>
34     <div class="col-12">
35       <label for="pass" class="form-label">Palavra-passe</label>
36       <input id="pass" name="pass" type="password" class="form-control">
37     </div>
38     <div class="col-12">
39       <button class="btn btn-primary" type="submit">Entrar</button>
40     </div>
41   </form>
42 </main>
43
44 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
45 </body>
46 </html>
```

## A.5. capital.html: Capital Lookup

```
1 <!DOCTYPE html>
2 <html lang="pt">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>World Capitals - Consultar Capital</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" re
8 </head>
9 <body>
10   <nav class="navbar navbar-expand-md bg-light border-bottom">
11     <div class="container">
12       <a class="navbar-brand" href="index.html">World Capitals</a>
13       <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-targ
14         <span class="navbar-toggler-icon"></span>
15       </button>
16       <div id="nav1" class="collapse navbar-collapse">
17         <ul class="navbar-nav ms-auto gap-2">
18           <li class="nav-item"><a class="nav-link" href="index.html">Países</a></li>
19           <li class="nav-item"><a class="nav-link active" href="capital.html">Consultar C
20           <li class="nav-item"><a class="nav-link" href="paises-editar.html">Gerir Países
21           <li class="nav-item"><a class="nav-link" href="login.html">Entrar</a></li>
```

## A. Complete Code: Static “World Capitals” Pages

```
22     </ul>
23   </div>
24 </div>
25 </nav>
26
27 <main class="container my-4">
28   <h1 class="display-6">Consultar Capital</h1>
29   <form class="row g-3 col-lg-6" method="GET" action="/capital">
30     <div class="col-12">
31       <label for="countrycode" class="form-label">Código do país</label>
32       <input id="countrycode" name="countrycode" type="text" maxlength="2" class="form-
33     </div>
34     <div class="col-12">
35       <button class="btn btn-primary" type="submit">Consultar</button>
36     </div>
37   </form>
38 </main>
39
40 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
41 </body>
42 </html>
```

## A.6. paises-editar.html: Country Management

```
1 <!DOCTYPE html>
2 <html lang="pt">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>World Capitals - Gerir Países</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" re
8 </head>
9 <body>
10   <nav class="navbar navbar-expand-md bg-light border-bottom">
11     <div class="container">
12       <a class="navbar-brand" href="index.html">World Capitals</a>
13       <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-targ
14         <span class="navbar-toggler-icon"></span>
15       </button>
16       <div id="nav1" class="collapse navbar-collapse">
17         <ul class="navbar-nav ms-auto gap-2">
18           <li class="nav-item"><a class="nav-link" href="index.html">Países</a></li>
19           <li class="nav-item"><a class="nav-link" href="capital.html">Consultar Capital<
20           <li class="nav-item"><a class="nav-link active" href="paises-editar.html">Gerir
21           <li class="nav-item"><a class="nav-link" href="login.html">Entrar</a></li>
22         </ul>
23       </div>
24     </div>
```



## A. Complete Code: Static “World Capitals” Pages

```
25 </nav>
26
27 <main class="container my-4">
28   <h1 class="display-6">Gerir Países</h1>
29
30   <h2 class="h5 mt-4">Inserir país</h2>
31   <form class="row g-3 col-lg-6" method="POST" action="/countryEdit">
32     <input type="hidden" name="operation" value="insert">
33     <div class="col-12">
34       <label for="code-i" class="form-label">Código</label>
35       <input id="code-i" name="code" type="text" maxlength="2" required class="form-control">
36     </div>
37     <div class="col-12">
38       <label for="name-i" class="form-label">Nome</label>
39       <input id="name-i" name="name" type="text" required class="form-control">
40     </div>
41     <div class="col-12">
42       <label for="capital-i" class="form-label">Capital</label>
43       <input id="capital-i" name="capital" type="text" required class="form-control">
44     </div>
45     <div class="col-12">
46       <button class="btn btn-success" type="submit">Inserir</button>
47     </div>
48   </form>
49
50   <h2 class="h5 mt-4">Atualizar país</h2>
51   <form class="row g-3 col-lg-6" method="POST" action="/countryEdit">
52     <input type="hidden" name="operation" value="update">
53     <div class="col-12">
54       <label for="code-u" class="form-label">Código</label>
55       <input id="code-u" name="code" type="text" maxlength="2" required class="form-control">
56     </div>
57     <div class="col-12">
58       <label for="name-u" class="form-label">Nome</label>
59       <input id="name-u" name="name" type="text" required class="form-control">
60     </div>
61     <div class="col-12">
62       <label for="capital-u" class="form-label">Capital</label>
63       <input id="capital-u" name="capital" type="text" required class="form-control">
64     </div>
65     <div class="col-12">
66       <button class="btn btn-warning" type="submit">Atualizar</button>
67     </div>
68   </form>
69
70   <h2 class="h5 mt-4">Remover país</h2>
71   <form class="row g-3 col-lg-6" method="POST" action="/countryEdit">
72     <input type="hidden" name="operation" value="delete">
73     <div class="col-12">
74       <label for="code-d" class="form-label">Código</label>
75       <input id="code-d" name="code" type="text" maxlength="2" required class="form-control">
```

### A. Complete Code: Static “World Capitals” Pages

```
76     </div>
77     <div class="col-12">
78         <button class="btn btn-danger" type="submit">Remover</button>
79     </div>
80 </form>
81 </main>
82
83 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
84 </body>
85 </html>
```

# B. JWT Token Signing

## B.1. Introduction

[Chapter 8](#) showed how a JWT (see [Section 8.3](#)) is signed and verified, without ever going into detail on **which algorithm** is used, or **what security guarantees** that signature actually offers. This appendix looks specifically at that question: token signing.

## B.2. HMAC: a Shared Secret Key

The algorithm used in [Chapter 8](#), HS256, is the combination of **HMAC** (*Hash-based Message Authentication Code*) with the **SHA-256** cryptographic hash function. It is a **symmetric** algorithm: the same secret key is used both to sign and to verify a *token*.

That key is never generated on the client, nor does it ever travel to it: it is created and stored exclusively on the server (typically once, during the application's initial setup), and never leaves it. What the client receives, and later resends, is always the already-signed *token* — the result of using the key — never the key itself.

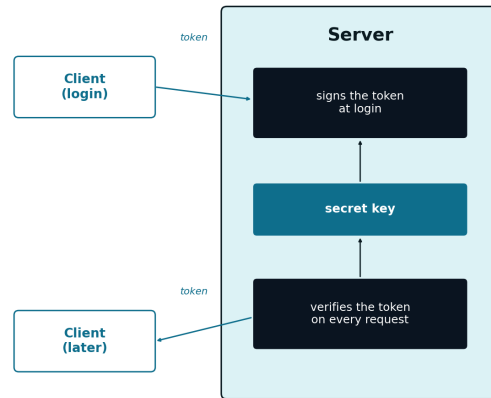
This works well in exactly the situation presented throughout this course: **a single server** (or several instances of that same server, all with access to the same secret key, see [Section 8.5](#)) is, at the same time, the one that issues *tokens* and the one that validates them. The key never leaves that trust boundary.

The drawback of HMAC appears when that boundary stops making sense: if an external system — one that should not have the authority to **issue** valid *tokens* — needed to **validate** them, it would have to receive the same secret key used to sign them. At that point, it stops being just a verifier: it also becomes capable of forging new, valid *tokens*, for any user.

## B.3. RSA and ECDSA: a Key Pair

An **asymmetric** algorithm solves this problem by splitting the two capabilities into two distinct keys: a **private key**, known only to whoever issues *tokens*, and a corresponding **public key**, which can be freely distributed to whoever needs to validate them. What the

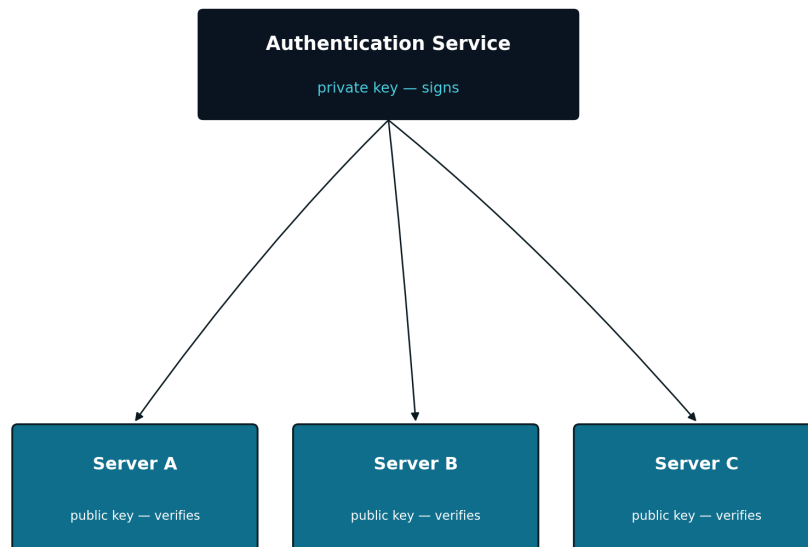
## B. JWT Token Signing



**Figure B.1.:** In a symmetric scheme (HMAC), the same server holds the single secret key that exists, and uses it both to sign the token at login and to verify it on every subsequent request.

private key signs can only be verified with the corresponding public key — but it is not possible to **sign** anything new using only the public key.

The two most common asymmetric algorithms in JWT are RS256 (RSA with SHA-256) and ES256 (*Elliptic Curve Digital Signature Algorithm* with SHA-256) — the latter more recent, with smaller keys and faster verification for an equivalent security level, increasingly preferred over RSA in new systems.



**Figure B.2.:** In an asymmetric scheme (RSA/ECDSA), only the Authentication Service holds the private key and signs new tokens; the corresponding public key can be freely distributed to any server that only needs to validate, without ever being able to issue tokens on another's behalf.

This design is the usual one in systems made up of several independent services (sometimes called *microservices*): a single Authentication Service, responsible for validating credentials and issuing *tokens*, and several other services, each validating those *tokens* completely on its own, without ever communicating with each other, or trusting each other beyond the shared

public key.

## B.4. Choosing Between HMAC and Asymmetric Signing

|                                   | HMAC (HS256)                                         | Asymmetric (RS256/ES256)                    |
|-----------------------------------|------------------------------------------------------|---------------------------------------------|
| Number of keys                    | One, shared                                          | Two: private (issues) and public (verifies) |
| Who can issue valid <i>tokens</i> | Anyone with the key                                  | Only whoever has the private key            |
| Who can validate <i>tokens</i>    | Anyone with the key                                  | Anyone with the public key                  |
| Typical scenario                  | One server (or trusted cluster) issues and validates | Several independent services only validate  |
| Performance                       | Faster, simpler                                      | Slower, larger keys (except ECDSA)          |

For the “World Capitals” application used throughout this course — a single server, responsible both for issuing and for validating its own *tokens* — HMAC is a perfectly adequate choice, which is why [Chapter 8](#) uses it as its example. The choice would only start to favour an asymmetric algorithm if the application grew into several independent services, each of which should be able to validate, but not issue, access credentials.

## B.5. The `alg:none` Attack

The *header* of a JWT itself declares which algorithm was used for the signature (see [Section 8.3](#)). The original JWT specification allows, among the possible values for that field, the value `none` — intended for cases where integrity is already guaranteed by some other mechanism, and no signature is even needed.

A poorly implemented verifier, one that blindly trusts the algorithm declared in the *header* instead of enforcing an expected algorithm, can be fooled by a *token* like this one:

```
1 {"alg": "none", "typ": "JWT"}
```

followed by an arbitrary *payload* and an empty signature. If the verification code does not explicitly reject the `none` algorithm, an attacker can build, by hand, a *token* with any *payload* they want — for example, `{"sub": "admin", "role": "admin"}` — without needing to know any key at all, and the server accepts it as legitimate.

It is exactly this attack that motivates the recommendation already made in [Chapter 8](#): a well-written library, such as JJWT, requires the expected algorithm to be stated explicitly in the verification code, and rejects any *token* that declares a different algorithm — including `none` — regardless of what the *token* itself claims about itself.

## B.6. Where to Store the Token on the Client

Everything covered so far protects the *token* against forgery — but not against theft (see Section 8.2): whoever holds it can use it freely, as if they were its rightful owner, until it expires. Where the client stores it between requests determines what kind of theft it is exposed to.

**Storing it in the *browser's localStorage*** Accessible to any JavaScript code running on the page — including, unfortunately, a malicious *script* injected through a *Cross-Site Scripting* (XSS) vulnerability. If such a vulnerability exists in the application, an attacker can read the *token* directly and resend it, impersonating the user. On the other hand, it is not automatically sent by the *browser* in requests to other sites, so it is not exposed to *Cross-Site Request Forgery* (CSRF)

**HttpOnly cookie** Marked with the `HttpOnly` attribute, a *cookie* becomes inaccessible to the page's own JavaScript — not even a malicious *script* can read it, which neutralises theft via XSS. In exchange, the *browser* automatically resends *cookies* on any request to the domain, even when that request is triggered by a page from another site — reopening the door to CSRF, which then has to be mitigated separately (for example, with the `SameSite` attribute)

|                                            | localStorage | HttpOnly cookie                   |
|--------------------------------------------|--------------|-----------------------------------|
| Accessible to JavaScript                   | Yes          | No                                |
| Vulnerable to XSS theft                    | Yes          | No                                |
| Automatically resent by the <i>browser</i> | No           | Yes                               |
| Vulnerable to CSRF                         | No           | Yes, requires separate mitigation |

Neither option is risk-free — each trades one kind of vulnerability for another. The model followed in Chapter 8 (*token* resent manually in the `Authorization` header, see Section 8.7) assumes the *token* is accessible to the page's own JavaScript, which points to `localStorage`, and is the most common approach in modern single-page applications. In that model, preventing XSS stops being just a general best practice: it becomes an essential condition for the security of the authentication mechanism itself — even though preventing XSS itself falls outside the scope of this appendix.

## B.7. Minimum Best Practices

- **Never blindly accept the *token's* declared algorithm** — the verification code must explicitly enforce the expected algorithm (see Section B.5)
- **The secret (HMAC) or private (RSA/ECDSA) key must never live in the source code** — it should be read from an environment variable or a dedicated secrets manager, never *committed* to a repository
- **A short or predictable key defeats the algorithm's security** — the recommended minimum length depends on the algorithm, but a short, memorable *string* is never appropriate
- **A short expiry** (the `exp claim`, see Section 8.4) limits the damage from a compromised *token*, even with no additional mechanism
- **HTTPS is always mandatory** — even a correctly signed *token* is a valid credential if it is intercepted in transit; the signature protects against forgery, not against network eavesdropping

### *B. JWT Token Signing*

These practices are a starting point, not an exhaustive treatment of Web application security.

# C. Development Environment Setup

## C.1. Introduction

This appendix gathers, in a single place, all the tools needed throughout this course, with their installation links, official documentation, and possible alternatives.

The instructions and links below apply equally to **macOS, Windows and Linux** — every tool listed has a version for all three systems. Where a step depends on the operating system, this is explicitly flagged.

## C.2. The Tools for This Course

**JDK — *Java Development Kit*** The Java language and the tools to compile and run it — the foundation for everything else. The [Eclipse Temurin](#) distribution is recommended: free, open source, maintained by the *Eclipse Adoptium Working Group*, and available for all three operating systems. Alternative: [Oracle JDK](#), Oracle’s official build, under a different licence. Help: official documentation at <https://docs.oracle.com/en/java/>; for specific questions, the [java tag on Stack Overflow](#) is the first place to check.

**Apache Tomcat** The *Servlet Container* used throughout the course (see [Chapter 4](#)), installed on the server side. Available at <https://tomcat.apache.org/>. Alternatives exist (*Jetty*, or a full application server such as *WildFly*, already mentioned in [Chapter 4](#)), but they are not used in this course. Help: official documentation at <https://tomcat.apache.org/tomcat-10.1-doc/>; the [tomcat tag on Stack Overflow](#) for specific questions.

**Eclipse IDE for Enterprise Java and Web Developers** The development environment (IDE) used for server-side code (Servlets, Filters, JSP), with built-in support for Jakarta EE projects and for installing and managing Tomcat directly from within Eclipse. Downloaded from <https://www.eclipse.org/downloads/packages/> (choose exactly the “Enterprise Java and Web Developers” package, not the generic “Eclipse IDE for Java Developers” package, which does not include the Web project tools). Alternative: [IntelliJ IDEA](#) (the *Ultimate* edition, paid — but free for students on registration — includes equivalent Jakarta EE support; the free *Community* edition does not). Help: [Eclipse community forum](#); the [eclipse tag on Stack Overflow](#).



- VSCode or WebStorm** A lighter editor, used for client-side code (HTML, CSS, JavaScript, see [Chapter 2](#) and [Chapter 8](#)) — it does not need either of Eclipse’s two Java EE-specific supports. [VSCode](#) is free and cross-platform; [WebStorm](#) is paid (free for students on registration), more specialised in JavaScript. Help: official documentation at <https://code.visualstudio.com/docs> (VSCode) or <https://www.jetbrains.com/webstorm/documentation/> (WebStorm).
- A modern browser, with developer tools** Chrome, Firefox and Edge all work equally well; what matters is knowing how to open their developer tools (*DevTools*, F12 key or Cmd+Option+I on Mac), used throughout the course to inspect network requests and debug JavaScript (see [Chapter 8](#)). Help: <https://developer.chrome.com/docs/devtools/> (Chrome) or <https://firefox-source-docs.mozilla.org/devtools-user/> (Firefox).
- A REST client** A tool for building HTTP requests by hand (GET, POST, PUT, DELETE, with headers and body of your choosing), useful for testing a REST API directly (see [Chapter 7](#)) without depending on any *front-office* already built. A simple GET can always be tested directly in the *browser*, but POST, PUT and DELETE cannot. [Postman](#) is the best known; [Insomnia](#) is a lighter alternative; whoever prefers the command line can use `curl`, already included by default on macOS, on Linux, and on Windows 10 and later.
- MySQL or MariaDB** The relational database management system used from LAB4 onward ([Appendix G](#)), when the fixed data used in the earlier LABs is replaced with real data, accessed through JDBC. [MySQL Community Server](#) is the free, open-source edition of MySQL; [MariaDB Server](#) is a free, open-source alternative, directly compatible (same network protocol, same JDBC *driver*) — either one works for this course. To create and manage the tables without relying solely on the command line, it is also useful to install a graphical client, such as [MySQL Workbench](#) or [DBeaver](#) (cross-platform, compatible with both MySQL and MariaDB). Help: official documentation at <https://dev.mysql.com/doc/> (MySQL) or <https://mariadb.com/kb/en/documentation/> (MariaDB); the `mysql` and `mariadb` tags on Stack Overflow for specific questions.

### C.3. Required Tools

From the list above, what is, in fact, indispensable for this course:

| Tool                                               | Minimum version     | Note                                                                                                                                                                           |
|----------------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JDK                                                | 17 (LTS)            | Required by Tomcat 10.1 (see below)                                                                                                                                            |
| Apache Tomcat                                      | 10.1.x              | <b>Do not use the 9.x series:</b> this course’s code uses the <code>jakarta.servlet.*</code> package (see <a href="#">Chapter 6</a> ), which only exists from Tomcat 10 onward |
| Eclipse IDE for Enterprise Java and Web Developers | Any current version | Or the equivalent IntelliJ IDEA Ultimate                                                                                                                                       |
| VSCode or WebStorm                                 | Any current version | For client-side code                                                                                                                                                           |
| A modern <i>browser</i>                            | Any current version | With <i>DevTools</i>                                                                                                                                                           |
| MySQL or MariaDB                                   | Any current version | Only needed from LAB4 onward ( <a href="#">Appendix G</a> )                                                                                                                    |

A REST client (Postman, Insomnia, or `curl`) is recommended, but not strictly required: the

first LABs can be tested with simple HTML forms (see [Chapter 3](#)), with no dedicated client at all.

## C.4. Confirming the Installation

Before moving on to any LAB, it is worth confirming that each tool is correctly installed:

- **JDK:** in a terminal, `java -version` should return the installed version number, with no errors
- **Apache Tomcat:** starting Tomcat and visiting `http://localhost:8080` in a *browser* should show Tomcat’s own default welcome page — no custom code is needed, or expected, yet
- **Eclipse:** should open with no errors, and the project creation menu should include the “Dynamic Web Project” option — a sign that Jakarta EE support is correctly installed
- **VSCode or WebStorm:** should open with no errors; creating any `.html` file and opening it directly in the *browser* confirms that the essentials already work, with no server needed
- **MySQL or MariaDB:** starting the server and connecting to it — via the command line (`mysql -u root -p`) or through the chosen graphical client — should work with no errors; no database of your own needs to exist yet, just confirm that the server responds

# D LAB1: First Endpoint and Simulated DAO

## D.1. Introduction to the Practical LABs

The practical LABs of this course build, step by step, a complete REST API — the “World Capitals API”. LABs 1 to 5 build the entire *backend*: each one is tested directly in the *browser* or with a REST client (see [Appendix C](#)), with no JavaScript at all. Only in [Appendix I](#), once the *backend* is finished, is that same *backend* connected to a *front-office* in JavaScript, through `fetch()` (see [Chapter 8](#)) — the same separation between the three pieces already presented in [Chapter 1](#).

The visual pages of this same application (country listing, login, capital lookup, CRUD forms) were already built in HTML/CSS/Bootstrap in [Chapter 2](#), with field names aligned with the API’s, and are available as complete, ready-to-copy files in [Appendix A](#). That is the code to which [Appendix I](#) will add the necessary `fetch()`.

## D.2. LAB1: First Endpoint and Simulated DAO

A good development environment setup is the foundation of any robust, maintainable application. Investing time up front in correctly preparing the tools and structuring the project saves, later on, countless hours of debugging and reorganising. This LAB guides the configuration of Eclipse and Tomcat, and the creation of the first *endpoint*, which will serve as the API’s “health check”.

### D.2.1. Step 1.1: Prepare the Development Environment

1. **Install the Eclipse IDE:** start by downloading and installing “Eclipse IDE for Enterprise Java and Web Developers” (see [Appendix C](#)). It is important to choose exactly this edition, since it comes pre-configured with the essential tools for this project
2. Install Apache Tomcat (see [Appendix C](#) for the required version), following the installation instructions, outside Eclipse
3. **Configure Apache Tomcat in Eclipse:**

- In Eclipse, go to the *Servers* perspective. If it is not visible, open it via **Window > Show View > Servers**
- Click the link to create a new server (“*No servers are available. Click this link to create a new server...*”)
- In the window that appears, expand the **Apache** folder and select the installed Tomcat version
- Follow the wizard’s steps, pointing to the folder where Tomcat was installed
- Once finished, the Tomcat server appears in the *Servers* perspective, ready to use

### D.2.2. Step 1.2: Create the *Dynamic Web Project*

1. From the Eclipse menu, select **File > New > Dynamic Web Project**
2. In the project creation window, set the **Project name** to **CapitaisAPI**
3. Confirm that the **Target runtime** is set to the Apache Tomcat server configured in the previous step
4. Click **Finish** to create the project
5. Once the project is created, it is good practice to organise the source code into packages: in the *Project Explorer*, expand the “Java Resources” section and locate the source folder (usually **src** or **src/main/java**). Right-click that folder, **New > Package**, and create the package **capitaisapi.servlets**

### D.2.3. Step 1.3: Develop the First *Endpoint* (*Servlet*)

The first *endpoint* is a simple status checker, to confirm that the API is working correctly.

1. Right-click the **capitaisapi.servlets** package just created, **New > Servlet**
2. In the window that opens, keep all fields at their pre-filled value, and name the class **StatusServlet**. Click **Finish**
3. Eclipse generates the class skeleton. Map this *Servlet* to the **/status URL** through the **@WebServlet** annotation, directly above the class declaration (it becomes **@WebServlet("/status")**)
4. Test the *Servlet*: in the file explorer, right-click the file containing the *Servlet*, **Run As > Run on Server**. Choose the Tomcat server of the same version used when creating the *Servlet*, and **Finish**. Within a few seconds, the *Servlet*’s result should appear in a *browser*
5. Create a new *Servlet*, following the same process, in the same package, called **CountryCapital**, and change the **doGet** method to return a simple JSON message. This method is invoked whenever an HTTP **GET** request is made to the **/countrycapital URL**. To write this method according to HTTP requirements and good programming practice (see [Chapter 4](#)), replace all of **doGet()**’s code with this:

```
1 // Inside the doGet method of the CountryCapital Servlet
2 response.setContentType("application/json"); // sets the content type to JSON
3 PrintWriter out = response.getWriter(); // obtains the output stream
4
5 // Manual construction of the JSON String
6 String jsonResponse = "{\"country\": \"Portugal\", \"capital\": \"Lisboa\", \"ccode\": \"\"";
7 out.print(jsonResponse); // sends the structured data to the client
8 out.flush();
```

`PrintWriter` needs to be imported, since it is not included in the *Servlet*'s generated skeleton. Note that the JSON *String* contains quotes inside it — so the quotes inside the Java *String* have to be written as `\` (escaped). Do not use single quotes inside the *String*, since they are not valid in JSON.

#### D.2.4. Step 1.4: Deploy and Test the Application

1. Test the *Servlet* again, exactly as in Step 1.3 (point 4). Eclipse compiles, deploys and starts the application on the server
2. It is also possible to test by right-clicking the project name (`CapitaisAPI`) — this opens a *browser* (or a testing tool). Enter the following *URL*:

```
http://localhost:8080/CapitaisAPI/countrycapital
```

3. The following response should appear on screen:

```
{"country": "Portugal", "capital": "Lisboa", "ccode": "PT"}
```

confirming that the first *endpoint* is working correctly. While the *Servlet* is running on the server, the *browser* can be closed and reopened, entering the same *URL*, and the result stays the same.

#### D.2.5. Step 1.5: Implement the Database Interface via a DAO

In a “real-life” application, dynamic content is normally generated from data coming from a database. The link between the Web application and the database can be made through a *Data Access Object* (DAO, see [Chapter 4](#)), which hides the database's implementation details from the Web application. This application will use a DAO to interface between the Web application and the database.

Since there is no access to a real database yet, this stage of the project will use a DAO that **simulates** access to the database (later, in LAB4, this DAO will be changed to query a real database). Note that, from the Web application's point of view, there is no difference at all between a DAO that fetches data from a real database and one that has the data fixed directly in the code.

Create a new class, called `DBDAO`, in the `capitaisapi.dao` package, and write three public methods in it:

- `List<String> getCountryCodes();`
- `String getCountryName(String countryCode);`
- `String getCapitalName(String countryCode);`

The first method returns the list of available country codes (PT, FR, EN, etc.), the second returns the country name associated with a code, and the last returns the capital name associated with a country code.

For this LAB, fix all return values directly in the `DBDAO` class's code. Use a `HashMap<String, String>` to hold the fixed values of country codes and capital names, and a `List` to hold the list of country codes.

By default, the `/countrycapital` *Servlet* should show the capital of the first country in the list returned by the DAO's `List<String> getCountryCodes()` method.

### D.2.6. Step 1.6: Create the Data Model (POJO)

To represent the application's data in a structured way, *POJOs* (*Plain Old Java Objects*) are used. A POJO is a data type, just like `String` or `Integer`, and so it helps guarantee that a given piece of information always follows the same rules. If a POJO is used as a method's return value, that value comes to respect what is defined in the POJO — in other words, a POJO is a **data model**. It is created already at this stage, so that the main pieces are defined, even though the POJO will only actually be used further on.

1. Create a new class called `CountryModel` in the `capitaisapi.servlets` package. This will be the POJO
2. Add private attributes to represent a country's data: `codigo`, `nome` and `capital`. Later, if convenient, others can be added, such as population or area
3. Generate the *getters* and *setters* for all attributes: right-click a blank line below the attribute definitions, **Source > Generate Getters and Setters** — Eclipse generates the code automatically

### D.2.7. Step 1.7: Develop the Capitals *Endpoint*

A new *Servlet* is now created that uses the DAO to return dynamic data. For testing purposes, the DAO should not return all of a country's information (code, name and capital) — the POJO is only used when it is necessary to return a country's complete information.

1. Create a new *Servlet* called `CapitalServlet`, in the `capitaisapi.servlets` package, mapped to the `/capital` URL
2. In the `doGet` method, implement the logic to:
  - Read the `countrycode` parameter from the request (`request.getParameter("countrycode")`)
  - Instantiate `DBDAO` and call `getCapitalName()` with the received country code
  - Check whether the country was found: if so, build a JSON *String* with the country code and the capital name; if not, return an appropriate error response. If the `countrycode` parameter is null or empty, return 400 (missing parameter, see [Chapter 3](#)); if `countrycode` has an unknown value, return 404 (country not found). Use `response.setStatus(HttpServletResponse.SC_BAD_REQUEST)` for 400, and `response.setStatus(HttpServletResponse.SC_NOT_FOUND)` for 404
  - Set the Content-Type to `application/json`, and send the JSON *String* as the response

### D.2.8. Step 1.8: Develop the Countries *Endpoint*

1. Create a new *Servlet* `CountryServlet`, in the same package as the previous ones, mapped to `/country`
2. In `DBDAO`, add a new method `CountryModel getCountry(String countrycode)`, meant to return all of a country's data (code, country name and capital name)
3. Using a structure similar to `CapitalServlet`'s, in the `doGet` method, obtain a country's complete data and return the complete JSON structure, properly validated by the `CountryModel` POJO

Note that the *Servlet* no longer manipulates isolated data (loose *Strings*), and instead works with a structured object — which improves readability, maintainability and data validation. The `getCountry()` method in `DBDAO` now centralises the aggregation of the data and encapsulates it in a structured model (POJO), reducing the *Servlet*'s responsibility.

# E.

## LAB2: Login and Session

### E.1. LAB2: Login and Session

Security is a fundamental concern in any Web application. Since this is a REST API, redirects are not used as in a traditional Web application — all communication with the client happens through HTTP status codes and JSON bodies (see [Chapter 3](#)). In this LAB, a *login endpoint* is added to the “World Capitals API”, and the HTTP session (see [Chapter 5](#)) is used to keep track, on the server side, of the fact that a given user is authenticated.

#### E.1.1. Step 2.1: Create the *Login Endpoint*

1. In the `capitaisapi.servlets` package (already used in the previous LABs, see [Appendix D](#)), create a new *Servlet* called `LoginServlet`, mapped to the `/login URL`
2. Implement the `doPost` method — not `doGet` — since credentials should not travel visibly in the *URL*
3. This method should receive two request parameters: `user` and `pass`

#### E.1.2. Step 2.2: Implement the Authentication Logic

Add a new public method to the `DBDAO` class (see [Appendix D](#)):

```
1 public boolean validateUserCredentials(String user, String pass)
```

For this LAB, fix the valid credentials directly in the code: username `javaee`, password `asw2026`. The method compares the received values against these fixed values, and returns `true` or `false`.

In `LoginServlet`’s `doPost` method, read the two request parameters:

```
1 String user = request.getParameter("user");  
2 String pass = request.getParameter("pass");
```

### E.1.3. Step 2.3: Create and Manage the Session

If the credentials are valid, the *Servlet* should:

1. Create (or obtain) an HTTP session: `HttpSession session = request.getSession(true);`
2. Store the authenticated user's identity in the session: `session.setAttribute("UserName", user);`
3. Respond with HTTP 200 and a confirmation JSON body

Example of a successful response:

```
{“message”: “OK”, “user”: “javaee”}
```

### E.1.4. Step 2.4: Handle Authentication Failures

Not every request to `/login` corresponds to a successful *login*. There are two cases to handle:

**Missing parameter** — if `user` or `pass` come in null or empty, respond with HTTP 400 BAD REQUEST:

```
{“message”: “BAD REQUEST!”, “error”: “Missing parameter user or pass”}
```

**Invalid credentials** — if `validateUserCredentials()` returns `false`, respond with HTTP 403 FORBIDDEN:

```
{“message”: “FORBIDDEN!”, “error”: “Invalid login or password”}
```

Note that, in neither of these cases, does the *Servlet* redirect the client to another page — it is the client that decides what to do next, based on the status code and the JSON body received.

### E.1.5. Step 2.5: Create an HTML File to Test the *Login Endpoint*

To test the `/login endpoint` without relying on external tools, a simple HTML form is created — with no JavaScript, no `fetch()`. That topic is reserved for [Appendix I](#), once the *backend* is complete.

Create the file at `src/main/webapp/login.html` — outside WEB-INF, since content inside WEB-INF is not directly accessible from the *browser* (see [Appendix C](#)):

```
1 <form method="POST" action="login">
2   <input type="text" name="user" required>
3   <input type="password" name="pass" required>
4   <button type="submit">Login</button>
5 </form>
```

This file serves only as a test client for the *endpoint* — it is not the application's final *login* form (that one is already finished in [Appendix A](#)).



### E.1.6. Step 2.6: Protect an *Endpoint* with Session Verification

With *login* working, the */country endpoint* (the `CountryServlet` from [Appendix D](#)) is now protected, so that it only responds to requests from authenticated users.

It is worth distinguishing two concepts here (see [Chapter 6](#)): **authentication** — confirming who the user is — and **authorisation** — confirming whether that user has permission to access a given resource. This step is only about authentication: checking whether a valid session exists.

At the start of `CountryServlet`'s `doGet` method, add:

```

1 HttpSession session = request.getSession(false);
2
3 if (session == null || session.getAttribute("UserName") == null) {
4     response.setStatus(HttpServletResponse.SC_FORBIDDEN);
5     response.setContentType("application/json");
6     PrintWriter out = response.getWriter();
7     String jsonResponse = "{\"error\": \"Authentication required\"}";
8     out.print(jsonResponse);
9     return;
10 }
```

Note the use of `request.getSession(false)`: unlike what happens at *login* (`getSession(true)`), here a new session should not be created just to check whether it exists — if there is no session, `getSession(false)` returns `null`. It is essential not to forget the `return` after writing the error response: without it, the code would keep executing the rest of the method, even after already having sent a 403 response.

### E.1.7. How to Test

1. Access `/country?countrycode=PT` directly, without logging in first. Expected result: HTTP 403, with the body `{"error": "Authentication required"}`
2. Submit the `login.html` form with `javaee / asw2026`. Expected result: HTTP 200, with the body `{"message": "OK", "user": "javaee"}`
3. Repeat the request to `/country?countrycode=PT`. Expected result now: HTTP 200, with the country's data

This LAB leaves the session-verification logic scattered across every *Servlet* that needs protection — in [Appendix F](#), that logic is centralised in a single Filter.



## LAB3: AuthenticationFilter and Logout

### F.1. LAB3: AuthenticationFilter and Logout

In [Appendix E](#), the session check was written directly inside `CountryServlet`. This approach works, but does not scale well: if there were ten protected *endpoints*, the same block of code would have to be copied ten times. In this LAB, that logic is centralised in a Filter (`jakarta.servlet.Filter`, see [Chapter 6](#)), and a *logout endpoint* is also added.

#### F.1.1. Step 3.1: Create and Configure the Authentication Filter

1. Create a new class called `AuthenticationFilter`, in the `capitaisapi.filters` package
2. This class should implement the `jakarta.servlet.Filter` interface
3. Map the Filter to the *endpoints* that should be protected, through the `@WebFilter` annotation:

```
1 @WebFilter(urlPatterns = {"/country", "/capital"})
2 public class AuthenticationFilter implements Filter {
3     // ...
4 }
```

Note that this mapping is an explicit list of protected *URLs* — unlike a `/*`-style pattern with exceptions, here only the *endpoints* listed in `urlPatterns` go through the Filter. Without the `@WebFilter` annotation, the Filter remains defined as a class, but is never executed by the server.

#### F.1.2. Step 3.2: Implement the Verification Logic in the Filter

The method to implement is `doFilter(ServletRequest request, ServletResponse response, FilterChain chain)`. The parameters arrive with the generic type `ServletRequest/ServletResponse` — when auto-generated by Eclipse, they may appear with generic names like `arg0`, `arg1`, `arg2`; it is worth renaming them to something clearer. Before using them, they need to be cast to the HTTP types:

```

1  @Override
2  public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
3      throws IOException, ServletException {
4
5      HttpServletRequest httpRequest = (HttpServletRequest) request;
6      HttpServletResponse httpResponse = (HttpServletResponse) response;
7
8      HttpSession session = httpRequest.getSession(false);
9
10     if (session != null && session.getAttribute("UserName") != null) {
11         chain.doFilter(request, response);
12     } else {
13         httpResponse.setStatus(HttpServletResponse.SC_FORBIDDEN);
14         httpResponse.setContentType("application/json");
15         PrintWriter out = httpResponse.getWriter();
16         String jsonResponse = "{\"error\": \"Authentication required\"}";
17         out.print(jsonResponse);
18     }
19 }

```

The call to `chain.doFilter(request, response)` is what lets the request continue on to the target *Servlet* — if the session is not valid, that call should not be made, and the Filter responds directly with 403 FORBIDDEN.

### F.1.3. Step 3.3: Remove the Manual Verification from the *Servlet*

With the Filter now handling authentication, the verification block added in [Step 2.6](#) is no longer needed inside `CountryServlet`. Remove that block: the *Servlet* goes back to containing only its own *endpoint*-specific logic, with no access-control concerns.

This separation — business logic in the *Servlet*, access control in the Filter — makes the code cleaner and easier to maintain: to protect a new *endpoint*, it is enough to add its *URL* to `urlPatterns` in [Step 3.1](#), with no need to touch any other code.

### F.1.4. Step 3.4: Create the *Logout Endpoint*

1. Create a new *Servlet* called `LogoutServlet`, in the `capitaisapi.servlets` package, mapped to the `/logout` *URL*
2. Implement the `doGet` method:

```

1  HttpSession session = request.getSession(false);
2
3  if (session != null) {
4      session.invalidate();
5  }
6
7  response.setContentType("application/json");
8  PrintWriter out = response.getWriter();
9  String jsonResponse = "{\"message\": \"Logout OK\"}";
10 out.print(jsonResponse);

```

`session.invalidate()` ends the session on the server side — from this point on, any later request carrying the same JSESSIONID (see [Chapter 5](#)) no longer finds a valid session.

### F.1.5. Step 3.5: Integrated Testing

1. Access `/country?countrycode=PT` without logging in. Expected result: HTTP 403, `{"error": "Authentication required"}`
2. Log in through `login.html` ([Appendix E](#)) with `javaee / asw2026`. Expected result: HTTP 200 with the success response — and, at this point, confirm (for example, in the *browser's* developer tools) that a JSESSIONID *cookie* was automatically created
3. Repeat the request to `/country?countrycode=PT`. Expected result: HTTP 200, with the country's data
4. Access `/logout`. Expected result: HTTP 200, `{"message": "Logout OK"}`
5. Repeat the request to `/country?countrycode=PT` once more. Expected result: HTTP 403 again — *logout* invalidated the session

### F.1.6. Conclusion

The API's architecture is now more modular: authentication remains concentrated in the */login endpoint*, but authorisation — deciding whether a given request may proceed — is no longer scattered across the *Servlets*, and is now centralised in a single Filter. In [Appendix G](#), the fixed data used so far is replaced with real data, stored in a relational database.

# G. LAB4: JDBC and Complete CRUD

## G.1. LAB4: JDBC and Complete CRUD

After building a functional API, with session-based authentication (LABs 1 to 3, [Appendix D](#), [Appendix E](#), [Appendix F](#)), the next step is to replace the fixed data written directly in the code with real data, stored in a relational database.

### G.1.1. Step 4.1: Prepare the Database

Before touching any Java code, a relational database (MySQL or MariaDB) must first be created, with two tables.

**Table `countries`**, with the fields:

- `code` (VARCHAR, primary key)
- `name` (VARCHAR)
- `capital` (VARCHAR)

**Table `users`**, with the fields:

- `username` (VARCHAR, primary key)
- `password` (VARCHAR)

At this stage, purely to keep things simple, passwords may be stored as plain text. In a real application, they should be stored in *hashed* form (for example, SHA-256), never as plain text.

### G.1.2. Step 4.2: Configure JDBC and Update DBDAO

1. Add the MariaDB/MySQL JDBC *driver* to the `src/main/webapp/WEB-INF/lib` folder
2. Update the DBDAO class (see [Appendix D](#)) to:
  - Create a method for establishing the database connection
  - Replace the fixed data with SQL queries
  - Change or create the following methods, for countries:

```

1 public List<String> getCountryCodes()
2 public CountryModel getCountry(String code)
3 public boolean insertCountry(String code, String name, String capital)
4 public boolean updateCountry(String code, String name, String capital)
5 public boolean deleteCountry(String code)

```

### Example of changing a DBDAO method

The following example shows the `insertCountry()` method already adapted for database access, using *prepared statements* — the correct way to include values coming from outside the application in a SQL statement, without opening the door to SQL injection (see [Chapter 4](#)):

```

1 public boolean insertCountry(String code, String name, String capital) {
2     if (code == null || code.isBlank() ||
3         name == null || name.isBlank() ||
4         capital == null || capital.isBlank()) {
5         return false;
6     }
7
8     if (code.length() != 2) {
9         return false;
10    }
11
12    String sql = "INSERT INTO countries (code, name, capital) VALUES (?, ?, ?)";
13
14    try (Connection conn = getConnection();
15         PreparedStatement ps = conn.prepareStatement(sql)) {
16
17        ps.setString(1, code);
18        ps.setString(2, name);
19        ps.setString(3, capital);
20
21        int rows = ps.executeUpdate();
22        boolean success = (rows == 1);
23        return success;
24    } catch (SQLException e) {
25        e.printStackTrace();
26        return false;
27    }
28 }
29

```

The remaining methods (`getCountryCodes`, `getCountry`, `updateCountry`, `deleteCountry`) follow the same principle: a SQL statement of its own, executed through a `PreparedStatement`, inside a `try-with-resources` block that automatically closes the connection.

3. Also implement the following methods, for users:

```

1 public boolean createUser(String username, String password)
2 public boolean validateUserCredentials(String username, String password)

```

From now on, *login* validation no longer compares against fixed values in the code, and instead queries the `users` table.

### G.1.3. Step 4.3: Update the *Login* Endpoint

`LoginServlet` ([Appendix E](#)) no longer uses fixed credentials in the code. The `validateUserCredentials()` method must:

- Query the `users` table
- Confirm that the user exists
- Validate the password

**Security note:** at this stage, for simplicity, direct comparison (plain text) may be used. Later, this should be implemented with *hashing* instead, both for storage and for validation.

### G.1.4. Step 4.4: Create a CRUD Endpoint Using POST and the operation Parameter

To insert and edit countries, a single *Servlet* is created, `CountryEditServlet`, mapped to the `/countryEdit` URL through `@WebServlet("/countryEdit")`, responsible for all changes to the `countries` table.

In the `doPost` method, the *Servlet* receives the `operation` parameter, with one of the values `insert`, `update` or `delete`. Depending on the value:

- Validate the required parameters
- Invoke the corresponding DBDAO method
- Return JSON and the appropriate status code, according to the table:

| Action              | Expected status code                                                     |
|---------------------|--------------------------------------------------------------------------|
| <code>insert</code> | 201 ( <code>SC_CREATED</code> )                                          |
| <code>update</code> | 200 ( <code>SC_OK</code> )                                               |
| <code>delete</code> | 200 ( <code>SC_OK</code> )                                               |
| <code>error</code>  | 400 ( <code>SC_BAD_REQUEST</code> ) or 404 ( <code>SC_NOT_FOUND</code> ) |

#### Example code for inserting a country

In the `doPost()` method, first read the value of the `operation` parameter:

```
1 String operation = request.getParameter("operation");
```

Then use a `switch` statement to invoke the corresponding DBDAO method:

```
1 switch (operation) {
2     case "insert":
3         String code = request.getParameter("code");
4         String name = request.getParameter("name");
5         String capital = request.getParameter("capital");
6
7         boolean inserted = dao.insertCountry(code, name, capital);
```

```

8
9     if (inserted) {
10         response.setStatus(HttpServletResponse.SC_CREATED);
11         String jsonResponse = "{\"message\": \"Country inserted successfully\"}";
12         out.print(jsonResponse);
13     } else {
14         response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
15         String jsonResponse = "{\"error\": \"Insert failed\"}";
16         out.print(jsonResponse);
17     }
18     break;
19
20     case "update":
21         // follows the same structure as the "insert" case: read the parameters,
22         // invoke dao.updateCountry(...), and respond with SC_OK or the appropriate error
23         break;
24
25     case "delete":
26         // follows the same structure: read the "code" parameter, invoke
27         // dao.deleteCountry(code), and respond with SC_OK or SC_NOT_FOUND
28         break;
29 }

```

The `update` and `delete` cases follow exactly the same structure as the `insert` case: read the parameters required by the request, invoke the corresponding DBDAO method, and respond with the appropriate status code from the table above.

### G.1.5. Step 4.5: Create HTML Forms for Testing

Create simple HTML files inside `src/main/webapp/`, just as was already done for *login* in [Appendix E](#). For example, the file to test the `insert` operation:

```

1 <form method="POST" action="countryEdit">
2   <input type="hidden" name="operation" value="insert">
3   <input type="text" name="code" maxlength="2" required>
4   <input type="text" name="name" required>
5   <input type="text" name="capital" required>
6   <button type="submit">Insert Country</button>
7 </form>

```

The hidden field (`type="hidden"`), the form's first line, is what sends the `operation` parameter via POST to the *Servlet* — the user does not see it, nor edit it.

Implement similar forms for every operation, for both the `countries` table and the `users` table. These files serve only as test clients for the API.

### G.1.6. Step 4.6: Protect the Endpoints with the Filter

`AuthenticationFilter` ([Appendix F](#)) must now also protect:



- `/country`
- `/countryEdit`
- (optionally) `/capital`

Simply add these *URLs* to `urlPatterns` in the Filter's `@WebFilter`. The user-creation endpoint can be left unprotected, for testing purposes only.

#### G.1.7. Step 4.7: Integrated Testing

1. Create a user
2. *Log in*
3. Insert a new country
4. Look up that country through `/country`
5. Update the country
6. Remove the country
7. *Log out*
8. Confirm that access is denied again (403)

#### G.1.8. Conclusion

In this LAB, the API starts using real data, stored in a relational database. The application stops being a static prototype and becomes a dynamic system, with data persistence and complete CRUD operations. This step lays the groundwork for future security improvements (password *hashing*), stricter validation, and a stateless architecture — the subject of [Appendix H](#).

# H LAB5 (Optional): Stateless Authentication with JWT

## H.1. LAB5 (Optional): Stateless Authentication with JWT

In LABs 1 to 4, server-side stateful authentication was implemented (`HttpSession`). This model is common in traditional Java Web applications, and is a good way of clearly understanding authentication and authorization.

In this optional LAB, the project evolves towards a modern, stateless REST API architecture: `HttpSession` and `JSESSIONID` are no longer used, the server stops keeping any per-user state between requests, and authentication is instead done with a JWT (*JSON Web Token*, see [Chapter 8](#) and [Appendix B](#)), sent with every protected request. This LAB is optional, aimed at anyone who wants to go further and understand how modern APIs typically authenticate clients.

**Objectives:** by the end of this LAB, it should be possible to explain the difference between stateful (session) authentication and stateless (*token*) authentication; implement JWT-based authentication; use the `HTTP Authorization` header with the `Bearer` scheme; validate a JWT inside a Filter; and correctly distinguish 401 from 403.

### H.1.1. Stateful vs. Stateless

**Session-based model (stateful)** — the model already used in the previous LABs: the server creates an `HttpSession`; Tomcat sends a `JSESSIONID` *cookie* to the *browser*; the server keeps state in memory (or in a shared session store). Works well for monolithic applications, but becomes harder to scale in distributed systems (multiple servers, load balancing, microservices, see [Chapter 8](#)).

**JWT model (stateless)** — the server keeps no session at all; the *login endpoint* generates a *token*; the client sends the *token* with every protected request; the server validates the *token* on every request. The authentication state (identity + expiry) is encoded inside the *token* itself; the application's data remains stored in the database.

### H.1.2. Step 5.1: Add a JWT Library

Generating and validating *JSON Web Tokens* requires an external library — the standard Java API includes no native JWT support. This LAB uses the **JWT** library (*Java JWT*, see [Appendix B](#)).

**Where to download it:** the library is available on Maven Central (search for `io.jsonwebtoken jjwt`). The following three artifacts need to be downloaded (example for version 0.11.5):

- `jjwt-api-0.11.5.jar`
- `jjwt-impl-0.11.5.jar`
- `jjwt-jackson-0.11.5.jar`

All three JAR files must be downloaded.

**Where to place the JAR files:** copy the three files to `src/main/webapp/WEB-INF/lib` — the same location already used earlier for the JDBC *driver* (see [Appendix G](#)).

After copying the files:

1. Stop the Tomcat server
2. Clean the project (Clean)
3. Restart the server

If everything is correct, the server starts up with no errors.

### H.1.3. Step 5.2: Create a Small JWT Utility Class

Create a new class `JwtUtil` (suggested package: `capitaisapi.security`). This class should centralise JWT operations:

- `String generateToken(String username)`
- `boolean validateToken(String token)`
- `String extractUsername(String token)` (optional, but useful)

Implementation requirements for this LAB:

- Sign the *tokens* with HMAC SHA-256 (HS256, see [Appendix B](#))
- Include at least: `sub` (*subject*) equal to the *username*, and `exp` (expiry date)
- Use a single shared secret key (fixed in the code for this LAB; in a real application, it should be read from outside the code, see [Appendix B](#))

Example implementation:

```
1 package capitaisapi.security;
2
3 import io.jsonwebtoken.*;
4 import io.jsonwebtoken.security.Keys;
5 import java.security.Key;
6 import java.util.Date;
7
8 public class JWTUtil {
9     // Secret key (must be at least 256 bits long for HS256)
```

```
10 private static final String SECRET =
11     "MySuperSecretKeyForJwtSigningMustBeLongEnough12345";
12
13 private static final long EXPIRATION_TIME = 1000 * 60 * 30; // 30 minutes
14
15 private static Key getSigningKey() {
16     return Keys.hmacShaKeyFor(SECRET.getBytes());
17 }
18
19 // Generates the token
20 public static String generateToken(String username) {
21     Date now = new Date();
22     Date expiryDate = new Date(now.getTime() + EXPIRATION_TIME);
23
24     JwtBuilder builder = Jwts.builder();
25     builder.setSubject(username);
26     builder.setIssuedAt(now);
27     builder.setExpiration(expiryDate);
28     builder.signWith(getSigningKey(), SignatureAlgorithm.HS256);
29
30     String token = builder.compact();
31     return token;
32 }
33
34 // Validates the token and returns the username if valid, or null otherwise
35 public static String validateToken(String token) {
36     try {
37         Claims claims = Jwts.parserBuilder()
38             .setSigningKey(getSigningKey())
39             .build()
40             .parseClaimsJws(token)
41             .getBody();
42         return claims.getSubject(); // username
43     } catch (JwtException | IllegalArgumentException e) {
44         return null;
45     }
46 }
47 }
```

Important: never store passwords inside the *token*.

#### H.1.4. Step 5.3: Change /login to Return a Token (No Session)

In this LAB, /login no longer creates a session:

- remove `request.getSession(true)`
- remove `session.setAttribute("UserName", ...)`

Instead, `LoginServlet` should:

1. Read `user` and `pass` from `doPost`

## H. LAB5 (Optional): Stateless Authentication with JWT

2. Validate the credentials with `DBDAO.validateUserCredentials(user, pass)`
3. If valid: generate a JWT with `JwtUtil.generateToken(user)`, and return HTTP 200 with a JSON body containing the *token*
4. If invalid: return HTTP 401 and a JSON error message

Example successful response:

```
{"token": "xxxxx.yyyyy.zzzzz"}
```

Example failure response:

```
{"error": "Invalid login or password"}
```

Status code rules (recommended): missing parameters → 400 (SC\_BAD\_REQUEST); invalid credentials → 401 (SC\_UNAUTHORIZED); valid credentials → 200 (SC\_OK).

### H.1.5. Step 5.4: Create a JWT Authentication Filter

Create a new Filter, for example `JwtAuthenticationFilter` (the same package as the previous Filter works fine, see [Appendix F](#)). Map it with `@WebFilter` to the *endpoints* that should be protected, for example `/country`, `/countryEdit` and, optionally, `/capital`.

Inside `doFilter`, the Filter should:

1. Read the `Authorization` header
2. Check that it exists and starts with `Bearer`
3. Extract the *token* after `Bearer`
4. Validate the *token*: the signature is correct, and the *token* is not expired
5. If valid, let the request continue: `chain.doFilter(request, response)`
6. If invalid or missing, block the request: return HTTP 401 (SC\_UNAUTHORIZED) with a JSON error body

Suggested error response:

```
{"error": "Invalid or expired token"}
```

Practical note: set `response.setContentType("application/json")` before writing the JSON.

### H.1.6. Step 5.5: Remove the Session-Based Filter (or Disable It)

At this stage, the old session-based model is no longer relied upon: remove the mapping of the old `AuthenticationFilter` (or stop using it), and remove any checks such as `request.getSession(false)` and `session.getAttribute("UserName")` from the *Servlets*. In this LAB, access control is done solely through *token* validation.

### H.1.7. Step 5.6: 401 vs. 403 in This Project

Always use these definitions: **401 Unauthorized** — the request is not authenticated (*token* missing, invalid, or expired); **403 Forbidden** — the request is authenticated, but the authenticated user does not have permission to access that resource.

This tutorial mostly uses **401**, because roles/permissions are not yet implemented. If roles are added later (for example, administrator vs. regular user), **403** becomes relevant. Recall: authentication answers “who are you?”; authorization answers “are you allowed to do this?” (see [Chapter 6](#)).

### H.1.8. Step 5.7: How to Test (*Browser* + *curl*/*Postman*)

Since *browsers* do not easily allow setting custom headers during normal navigation, testing a JWT is best done with *curl* or *Postman* (see [Appendix C](#)).

- 1) Log in and obtain a token:** send a **POST** request to `/login` with valid credentials (`javaee / asw2026`) and copy the *token* from the JSON response.
- 2) Call a protected endpoint with the token:** send a **GET** request to `/country?countrycode=PT`, with the header `Authorization: Bearer <token>`. Expected result: valid *token* → **HTTP 200** with JSON data; missing or invalid *token* → **HTTP 401** with a JSON error.

In this LAB, always send the *token* through the **Authorization** header. Do not store the JWT in *cookies* — that would, in practice, reintroduce implicit session-like behaviour.

- 3) Test expiry:** wait until the *token* expires (or set a very short expiry just for testing), and repeat the same request — **401** should now be obtained. After *login*, also confirm that the server does not create any **JSESSIONID** *cookie*: that is the confirmation that the application is, in fact, stateless.

### H.1.9. Conclusion

This optional LAB shows the natural evolution of the API across the five LABs: session-based (stateful) authentication → authorization logic centralised in a Filter → real persistence with a database → stateless authentication with JWT. With JWT, the API becomes easier to scale horizontally, and aligns with modern server-side architectures, especially when serving *SPAs* (*Single-Page Applications*) and mobile applications.



## LAB6: The Front-Office in JavaScript

### I.1. LAB6: The Front-Office in JavaScript

Up to this point, the *backend* built in LABs 1 to 5 was always tested directly — in the *browser*, with simple HTML forms, or with a REST client (see [Appendix C](#)). This final LAB connects, for the first time, the ready-made visual pages from [Appendix A](#) to that same *backend*, through `fetch()` (see [Chapter 8](#)) — the missing piece of JavaScript needed to close the “three pieces” introduced in [Chapter 1](#).

This LAB assumes the session-based *backend* built in LABs 1 to 4 ([Appendix D](#) to [Appendix G](#)), already with real data in a database (LAB4). Anyone who also completed the optional LAB5 (stateless authentication with JWT, [Appendix H](#)) will find, at the end of this section, a note on what changes in that case.

#### I.1.1. Step 6.1: Prepare the Project

1. Copy the four files from [Appendix A](#) (`index.html`, `login.html`, `capital.html`, `paises-editor.html`) to `src/main/webapp/`, inside the same Eclipse `CapitaisAPI` project already used in the previous LABs. These pages replace the “bare” test forms created throughout the previous LABs (`login.html` from LAB2, the `paises-editor.html` forms from LAB4) — from now on, the application has a single interface
2. Inside `src/main/webapp/`, create a `js/` folder
3. In each of the four `.html` files, add, before `</body>`, right after the *Bootstrap* `<script>`, a link to that page’s own JavaScript file (and, in all of them except `login.html`, also to the shared `js/auth.js` — see [Step 6.4](#)):

```
1 <script src="js/auth.js"></script>
2 <script src="js/countries.js"></script>
```

Since the pages are now served by Tomcat itself (rather than opened directly as a local file), `fetch()` requests can use simple relative *URLs*, with no leading slash — exactly the same pattern already used in the HTML forms from the previous LABs (`action="login"`, `action="countryEdit"`): they resolve automatically relative to the current page, within the

same *host* and the same context (CapitaisAPI), with no cross-origin (*CORS*) problem at all.

### I.1.2. Step 6.2: A Small Addition to the *Backend* — Listing the Available Codes

None of the previous LABs exposed a way for a client to discover *which* country codes are available — it is only possible to look up one country at a time, already knowing its code (`/country?countrycode=PT`). To build the home page's listing, one small additional *endpoint* is needed.

Create a new *Servlet*, `CountryCodesServlet`, in the `capitaisapi.servlets` package, mapped to the `/countrycodes` *URL*:

```

1 package capitaisapi.servlets;
2
3 import java.io.IOException;
4 import java.io.PrintWriter;
5 import java.util.List;
6
7 import capitaisapi.dao.DBDAO;
8
9 import jakarta.servlet.ServletException;
10 import jakarta.servlet.annotation.WebServlet;
11 import jakarta.servlet.http.HttpServlet;
12 import jakarta.servlet.http.HttpServletRequest;
13 import jakarta.servlet.http.HttpServletResponse;
14
15 @WebServlet("/countrycodes")
16 public class CountryCodesServlet extends HttpServlet {
17
18     @Override
19     protected void doGet(HttpServletRequest request, HttpServletResponse response)
20         throws ServletException, IOException {
21
22         DBDAO dao = new DBDAO();
23         List<String> codes = dao.getCountryCodes();
24
25         StringBuilder json = new StringBuilder("[");
26         for (int i = 0; i < codes.size(); i++) {
27             if (i > 0) {
28                 json.append(",");
29             }
30             json.append "\"" + codes.get(i) + "\"";
31         }
32         json.append("]");
33
34         response.setContentType("application/json");
35         PrintWriter out = response.getWriter();
36         String jsonResponse = json.toString();
37         out.print(jsonResponse);

```



```

38     }
39 }

```

This *Servlet* does nothing new on the data side — it reuses `getCountryCodes()`, already present in DBDAO since [Appendix D](#) — it just exposes, for the first time, that list to a client, in JSON.

Since this listing is, just like each country’s data, information reserved for authenticated users, add `/countrycodes` to the mapping of `AuthenticationFilter` ([Appendix F](#)):

```

1 @WebFilter(urlPatterns = {"/country", "/countrycodes", "/countryEdit"})

```

This LAB assumes that protection for `/capital` was not enabled (it stayed public, as allowed — “optionally” — in [Appendix G](#)): the `capital.html` page keeps working without *login*.

### I.1.3. Step 6.3: Authentication Without Reloading the Page

In `login.html`, add an `id` to the form and a message area right after it:

```

1 <form class="row g-3 col-lg-6" method="POST" action="/login" id="form-login">
2     ...
3 </form>
4 <div id="feedback" class="mt-3"></div>

```

Create `js/login.js`:

```

1 document.querySelector("#form-login").addEventListener("submit", function (event) {
2     event.preventDefault();
3
4     const username = document.querySelector("#user").value;
5     const password = document.querySelector("#pass").value;
6
7     const body = new URLSearchParams();
8     body.append("user", username);
9     body.append("pass", password);
10
11     fetch("login", {
12         method: "POST",
13         body: body
14     })
15     .then(function (response) {
16         const success = response.ok;
17         return response.json().then(function (data) {
18             return { success: success, data: data };
19         });
20     })
21     .then(function (result) {
22         const feedback = document.querySelector("#feedback");
23         if (result.success) {

```

```

24     markAuthenticated();
25     window.location.href = "index.html";
26   } else {
27     feedback.textContent = result.data.error || result.data.message;
28     feedback.className = "mt-3 text-danger";
29   }
30   });
31 });

```

`event.preventDefault()` stops the form's normal submission — from here on, it is `fetch()` that decides where the request goes, and the form's `action` attribute is no longer used. Note that there is no need to set the `Content-Type` header manually: when the request body is a `URLSearchParams`, the *browser* itself automatically sets `application/x-www-form-urlencoded`, the same format `LoginServlet` has expected since [Appendix E](#) (`request.getParameter()`).

The `markAuthenticated()` function, used here, comes from the next step — since `login.html` also loads `js/auth.js` (see [Step 6.1](#)), that function is already available.

#### I.1.4. Step 6.4: Show/Hide Based on Authentication State

In each of the four files, in the navigation bar block, replace the “Entrar” link line with these two:

```

1 <li class="nav-item"><a class="nav-link" href="login.html" id="nav-login">Entrar</a></li>
2 <li class="nav-item"><a class="nav-link d-none" href="#" id="nav-logout">Sair</a></li>

```

Create `js/auth.js`, shared by all pages:

```

1 function markAuthenticated() {
2   localStorage.setItem("authenticated", "yes");
3 }
4
5 function clearAuthentication() {
6   localStorage.removeItem("authenticated");
7 }
8
9 function isAuthenticated() {
10  return localStorage.getItem("authenticated") === "yes";
11 }
12
13 function updateNav() {
14   const loginLink = document.querySelector("#nav-login");
15   const logoutLink = document.querySelector("#nav-logout");
16
17   if (!loginLink || !logoutLink) {
18     return;
19   }
20
21   if (isAuthenticated()) {

```

```

22     loginLink.classList.add("d-none");
23     logoutLink.classList.remove("d-none");
24 } else {
25     loginLink.classList.remove("d-none");
26     logoutLink.classList.add("d-none");
27 }
28 }
29
30 document.addEventListener("DOMContentLoaded", function () {
31     updateNav();
32
33     const logoutLink = document.querySelector("#nav-logout");
34     if (logoutLink) {
35         logoutLink.addEventListener("click", function (event) {
36             event.preventDefault();
37             fetch("logout").then(function () {
38                 clearAuthentication();
39                 window.location.href = "index.html";
40             });
41         });
42     }
43 });

```

Note that the visible text of the two links — “Entrar” and “Sair” — stays in Portuguese: these links belong to the shared navigation bar first built in [Chapter 2](#) and reused throughout [Appendix A](#), and its screenshots were never retaken in this English edition.

This `localStorage` is only a convenience for the interface — it is not, in any way, real security. What still decides whether a request to `/country`, `/countrycodes` or `/countryEdit` is accepted is `AuthenticationFilter`, on the server side (see [Appendix F](#)). If someone opens the *browser*’s developer tools and manually changes this value, the worst that happens is seeing a button that should not be there — protected requests still get a 403 response from the server.

Since `países-editor.html` only makes sense for someone with an active session, add a simple check at the start of `js/countries-edit.js` (see [Step 6.7](#)):

```

1  if (!isAuthenticated()) {
2      window.location.href = "login.html";
3  }

```

### I.1.5. Step 6.5: List Countries on the Home Page

In `index.html`, give the table’s `<tbody>` an `id` and remove the three fixed example rows, since the table now becomes dynamically populated:

```

1  <tbody id="countries-table"></tbody>

```

Create `js/countries.js`:

```

1  if (isAuthenticated()) {
2    fetch("countrycodes")
3      .then(response => response.json())
4      .then(function (codes) {
5        const requests = codes.map(function (code) {
6          return fetch("country?countrycode=" + code).then(response => response.json());
7        });
8        return Promise.all(requests);
9      })
10     .then(function (countries) {
11       const table = document.querySelector("#countries-table");
12       countries.forEach(function (country) {
13         const row = document.createElement("tr");
14         row.innerHTML = "<td>" + country.code + "</td><td>" + country.name + "</td><td>"
15         table.appendChild(row);
16       });
17     });
18   } else {
19     const table = document.querySelector("#countries-table");
20     table.innerHTML = "<tr><td colspan='3'>Log in to see the list of countries.</td></tr>";
21   }

```

This step combines two ideas already introduced separately: the `fetch()` + DOM pattern from [Chapter 8](#), and `Promise.all()`, which waits for several *promises* in parallel (one per country) and only continues once all of them have finished — faster than requesting the countries one by one, in sequence.

### I.1.6. Step 6.6: Look Up a Capital

In `capital.html`, give the form ids and add a result area:

```

1  <form class="row g-3 col-lg-6" method="GET" action="/capital" id="form-capital">
2    ...
3  </form>
4  <div id="result" class="mt-3"></div>

```

Create `js/capital.js`:

```

1  document.querySelector("#form-capital").addEventListener("submit", function (event) {
2    event.preventDefault();
3
4    const code = document.querySelector("#countrycode").value;
5
6    fetch("capital?countrycode=" + code)
7      .then(function (response) {
8        const success = response.ok;
9        return response.json().then(function (data) {
10          return { success: success, data: data };
11        });
12      })

```

```

13     .then(function (result) {
14         const resultEl = document.querySelector("#result");
15         if (result.success) {
16             resultEl.textContent = "Capital of " + code + ": " + result.data.capital;
17             resultEl.className = "mt-3";
18         } else {
19             resultEl.textContent = result.data.error || "Country not found";
20             resultEl.className = "mt-3 text-danger";
21         }
22     });
23 });

```

### I.1.7. Step 6.7: Country CRUD

In `paises-editor.html`, give the three forms ids (`form-insert`, `form-update`, `form-delete`) and add a single message area, shared by all three:

```

1 <form class="row g-3 col-lg-6" method="POST" action="/countryEdit" id="form-insert">
2     ...
3 </form>
4     ...
5 <div id="feedback" class="mt-3"></div>

```

Create `js/countries-edit.js`:

```

1 if (!isAuthenticated()) {
2     window.location.href = "login.html";
3 }
4
5 function submitOperation(form, successMessage) {
6     form.addEventListener("submit", function (event) {
7         event.preventDefault();
8
9         const body = new URLSearchParams(new FormData(form));
10
11         fetch("countryEdit", {
12             method: "POST",
13             body: body
14         })
15             .then(function (response) {
16                 const success = response.ok;
17                 return response.json().then(function (data) {
18                     return { success: success, data: data };
19                 });
20             })
21             .then(function (result) {
22                 const feedback = document.querySelector("#feedback");
23                 if (result.success) {
24                     feedback.textContent = successMessage;
25                     feedback.className = "mt-3 text-success";

```

```

26     form.reset();
27   } else {
28     feedback.textContent = result.data.error || "Operation failed";
29     feedback.className = "mt-3 text-danger";
30   }
31 });
32 });
33 }
34
35 submitOperation(document.querySelector("#form-insert"), "Country inserted successfully.");
36 submitOperation(document.querySelector("#form-update"), "Country updated successfully.");
37 submitOperation(document.querySelector("#form-delete"), "Country removed successfully.");

```

`new FormData(form)` automatically collects every field with a `name` inside the submitted form — including the hidden `operation` field, already present in each of the three forms since [Appendix A](#) — with no need to list them one by one; `new URLSearchParams(...)` converts that collection directly into the format `CountryEditServlet` expects. The same `submitOperation()` function serves all three forms, because all three follow exactly the same request/response structure (see [Appendix G](#)) — only the success message shown changes.

### I.1.8. How to Test

1. Open `http://localhost:8080/CapitaisAPI/index.html` with no session started. The table should show the message asking to log in, and the navigation bar should show “Entrar”
2. Log in on `login.html`. Expected result: redirect to `index.html`, already with the table populated, and the navigation bar showing “Sair” instead of “Entrar”
3. Test `capital.html`: look up a valid country code and an invalid one, and confirm both messages
4. Test `paises-editor.html`: insert a new country, confirm it appears in `index.html`; update it; remove it
5. With no session started (for example, in an anonymous *browser* window), try to access `paises-editor.html` directly. Expected result: automatic redirect to `login.html`
6. Log out through the “Sair” link. Expected result: back to `index.html`, with no session again, showing “Entrar” in the navigation bar

### I.1.9. Conclusion

With this LAB, the cycle introduced in [Chapter 1](#) closes: the three pieces — Frontend, Middleware, Backend — stop being a theoretical map of the book, and become an actually working application. The *backend* from LABs 1 to 4 never needed to know who would consume it; this LAB confirms exactly that, connecting it to a client built from scratch, without changing the business logic or the authentication already implemented.

**Note, for anyone who completed the optional LAB5 ([Appendix H](#)):** if the *backend* used is the stateless one, based on JWT, the only change needed is in how the authentication state is stored and sent. Instead of relying on an automatic session *cookie*: also store the *token* returned by *login* (for example, alongside the authenticated flag itself, in `localStorage`); and send it with every protected request through the `Authorization: Bearer <token>` header (see [Appendix H](#) and [Chapter 8](#)), in `fetch()`:

## *I. LAB6: The Front-Office in JavaScript*

```
1 fetch("country?countrycode=" + code, {  
2   headers: { "Authorization": "Bearer " + token }  
3 })
```

Everything else — showing/hiding based on authentication, listing countries, the complete CRUD — stays exactly the same.